

Highlights

Automated and Configurable Undo for CRDT-Based Distributed Systems via Metaprogramming

Provakar Mondal and Eli Tilevich

- Introduces CAMEL, a transparent automated undo support for operation-based CRDTs
- Generates undo functionality automatically by means of metaprogramming
- Enhances code modularity by providing undo without modifying the CRDT source code
- Improves latency and memory efficiency over two state-of-the-art approaches
- Triggers undo based on configurable deterministic and probabilistic models

Automated and Configurable Undo for CRDT-Based Distributed Systems via Metaprogramming

Provakar Mondal and Eli Tilevich

*Software Innovations Lab, Virginia Tech, 220 Gilbert
Place, Blacksburg, 24061, Virginia, USA*

Abstract

Distributed nodes can access and modify replicated data using Conflict-free Replicated Data Types (CRDTs), which provide intuitive programming abstractions and enforce Strong Eventual Consistency (SEC) for conflict resolution. As a result of user errors, program bugs, or hardware malfunctions, a CRDT may be updated incorrectly, necessitating the undoing of the effects of executed operations. However, since many CRDT libraries lack built-in undo support, adding this capability requires modifying the library’s source code, which is difficult to implement in a modular and reusable manner. Consequently, programmers often add this functionality ad hoc, resulting in code that is difficult to understand, maintain, and reuse. To address this problem, this paper presents CAMEL, a metaprogramming-based approach that automatically generates and actuates undo functionality for existing operation-based CRDT libraries using simple metadata configurations, without requiring manual source code modifications. The configurations specify which operations undo each other and the conditions that trigger the undo procedures. Based on this metadata, CAMEL generates a sequence of update operations that reverses the specified updates while preserving system consistency. We have implemented CAMEL in JavaScript and evaluated it against two state-of-the-art approaches. To demonstrate CAMEL’s applicability, we apply it to three representative CRDTs of increasing complexity—Counter, Set, and Map—that span scalar, collection, and associative data structures commonly deployed in practice. Our evaluation demonstrates that CAMEL reduces latency by approximately 7%–12% and memory consumption by approximately 6%–26%, on average, while software quality metrics shows that CAMEL enhances modularity, effectively streamlining the complexity of

adding advanced features to distributed programming paradigms¹.

Keywords:

Conflict-free Replicated Data Types (CRDTs), Metaprogramming, Strong Eventual Consistency, Automated Error Recovery, Distributed Systems Reliability

1. Introduction

In modern distributed applications, data is replicated across sites to reduce access latency, increase availability, and provide fault tolerance [2, 3]. Because individual replicas can be modified independently, their states must be synchronized to ensure consistency. The Strong Eventual Consistency (SEC) model has proven effective for this purpose, guaranteeing that replicas that have received the same set of updates will eventually reach the same state, regardless of the order in which the updates are applied [4]. Conflict-free Replicated Data Types (CRDTs) achieve SEC without requiring coordination or complex conflict resolution [5]. They enable high availability by allowing local, independent updates, thereby guaranteeing that all replicas converge to the same state once all messages have been received. A CRDT exposes an interface of update and access methods, hiding the complexities of synchronization from the programmer.

Depending on how updates propagate across replicas, CRDTs are classified into two types: state-based (CvRDTs) and operation-based (CmRDTs). In state-based CRDTs, replicas merge entire states, whereas in operation-based CRDTs, replicas propagate individual update operations for replay by peers [6]. Real-world CRDT-based applications include SoundCloud, PayPal, Facebook’s OpenR, and Apple’s Notes [7]. While the inherent properties of CRDTs ensure resilience against network partitions and partial failures, SEC provides no remedy for application-level errors—such as incorrect user inputs, software bugs, or sensor malfunctions in IoT/Edge environments [8, 9]. Because these updates satisfy the internal consistency logic, they propagate across the system, resulting in a globally consistent but erroneous state.

¹This article is a revised and extended version of our prior paper, published in the 14th IEEE International Conference on Cloud Computing Technology and Science (CloudCom 2023) [1]

To recover from these errors, developers require an undo mechanism. However, as noted in recent studies, many CRDT libraries lack built-in undo support, and application programmers find implementing such functionality manually complex and confusing [10]. Existing state-of-the-art solutions often force developers to modify library source code or design new, augmented data types. This process is laborious and error-prone, often resulting in ad hoc code that is difficult to maintain or reuse [11].

To address this problem, we present CAMEL (**C**onfigurable, **A**utomated, and **M**etaprogramming-based **L**ifter), an approach that introduces undo functionality to existing *operation-based CRDT (CmRDT)* libraries using declarative metadata. By treating CRDT libraries as unmodified black boxes, CAMEL uses metaprogramming to intercept updates and automatically generate an Undo Script—a sequence of counter-operations that revert erroneous changes. A key feature of CAMEL is its ability to identify `no_op`—operations that do not change the CRDT state—thereby ensuring that the Undo Script targets only effective updates. Unlike prior state-of-the-art approaches, CAMEL adopts a modular layered architecture that separates undo generation (creating Undo Script) from actuation (deciding whether to undo), providing undo through well-defined component boundaries. It also provides deterministic (statistical) and probabilistic (ML-based) strategies for identifying when an undo is required. Our reference implementation of CAMEL targets JavaScript, one of the most widely used languages for CRDT libraries [12]. To demonstrate CAMEL’s applicability, we apply it to three representative CRDTs of increasing complexity—Counter, Set, and Map—that span scalar, collection, and associative data structures commonly deployed in practice [13].

This article revises and extends our preliminary work presented at CloudCom’23 [1] in several key dimensions. First, we prove that CAMEL’s intercepted operations and undo actuation preserve the system’s SEC. Second, we have expanded our evaluation to include a direct performance comparison against two state-of-the-art baselines. Third, we analyze software engineering metrics, such as Lines of Code (LoC) and modularity, to quantify the benefits of CAMEL’s modular layered architecture. Finally, we discuss how CAMEL infers `no_op` and achieves causal ordering in distributed settings.

The primary contributions of this paper are:

1. A novel metaprogramming approach for automatically generating and actuating undo functionality for operation-based CRDTs without mod-

ifying their source code.

2. A proof that CAMEL’s undo mechanisms—specifically its Undo Script generation, `no_op` detection, and causal queueing—maintain SEC of the underlying CRDT.
3. A reference implementation of CAMEL in JavaScript that utilizes JSON-based metadata and advanced code adaptation, applied to Counter, Set, and Map CRDTs across two popular third-party CRDT libraries.
4. An empirical evaluation comparing CAMEL with two state-of-the-art approaches, demonstrating improvements in performance and code modularity.

The remainder of this paper is organized as follows: Section 2 provides the background and motivates the problem; Section 3 presents our solution approach; Section 4 details CAMEL’s system design and key components; Section 5 describes the reference implementation; Section 6 presents our evaluation results; Section 7 discusses the derived insights; Section 8 positions this work with the related state of the art; and finally Section 9 concludes the paper with future work plans.

2. Background and Motivation

In this section, we first present the technical background and then motivate the need for this research.

2.1. Background

This work concerns metaprogramming-based undo support for operation-based CRDTs with a Strong Eventual Consistency (SEC) model. We introduce the key concepts next.

Conflict-free Replicated Data Types (CRDTs). CRDTs are data structures designed for distributed systems that allow multiple replicas to be updated independently and concurrently, while guaranteeing that all replicas eventually converge to the same state without requiring central coordination [5]. A CRDT is built around three fundamental components: (1) **state**, (2) **interface**, and (3) **update propagation** [14]. The **state** encodes the internal representation of the data type; CRDTs encompass a wide spectrum of data structures, from foundational ones such as counters, sets, and maps to

more sophisticated structures such as JSON documents [15]. The **interface** exposes operations that allow clients to read and modify the CRDT state. The **update propagation** mechanism disseminates local state changes to all other replicas, driving them toward eventual convergence. The two most common update propagation strategies in CRDT design are state-based and operation-based synchronization. *State-based CRDTs (CvRDTs)* propagate their entire local state to other replicas for synchronization, with each replica merging the received state using a merge function that must be commutative, associative, and idempotent to guarantee convergence [16].

Operation-based CRDTs (CmRDTs) disseminate a compact, encoded representation of each update operation to the remaining replicas, which then apply it locally [6]. Rather than exchanging full states, replicas communicate only the operations that change state, making this approach more bandwidth-efficient. CmRDTs require that their update operations be commutative, ensuring that, regardless of the order in which replicas receive and apply operations, all replicas eventually reach the same state. CAMEL applies exclusively to operation-based CRDTs, as their operation-level granularity supports the fine-grained undo functionality provided by CAMEL.

Strong Eventual Consistency (SEC). Eventual consistency is a widely adopted consistency model that prioritizes availability and partition tolerance by permitting temporary divergence between replicas [17]. As replicas independently process local updates, network delays and node failures may defer the propagation of those updates to other replicas. Nevertheless, once all participating replicas have received and applied the same set of updates, they will converge to an identical replicated state [18].

Let $R = \{R_1, R_2, \dots, R_n\}$ be the set of replicas, $U = \{u_1, u_2, \dots, u_m\}$ be the set of update operations, and $S_i(t)$ denote the state of replica R_i at time t . Let $\delta(u, S_i, t)$ denote the state modification induced by applying update u to replica R_i in state S_i at t , producing successor state $S'_i(t)$. If $S_i(t)' \neq S_i(t)$, u actually modifies the state, and if $S_i(t)' = S_i(t)$, u is a **no_op** on the state S_i (we formalize **no_op** in Definition 3.1). The system achieves eventual consistency if every update is eventually delivered to and applied by every replica:

$$\forall u \in U, \forall R_i \in R, \exists t \text{ such that } S'_i(t) = \delta(u, S_i, t).$$

As synchronization progresses, all replica states converge:

$$\forall R_i, R_j \in R, \quad \lim_{t \rightarrow \infty} S_i(t) \equiv S_j(t).$$

SEC extends this model by adding a determinism guarantee: any two replicas that have processed the same set of updates will be in identical states, irrespective of the order in which those updates were applied [4]. SEC is particularly well-suited for operation-based CRDTs, where the commutativity of update operations provides a natural mechanism for achieving deterministic convergence. Prominent distributed storage systems, including Amazon DynamoDB, Apache Cassandra, Riak, Firebase, and Voldemort, rely on eventual consistency to deliver scalable, highly available data management [19, 20].

Metaprogramming. Metaprogramming is a programming paradigm in which a program can treat other programs or itself as data, enabling it to read, generate, analyze, or transform code [21, 22, 23]. Rather than static hardcoding, metaprogramming enables a system to adapt its functionality dynamically in response to external specifications or runtime conditions. Common metaprogramming mechanisms include reflection, which enables a program to inspect and modify its own structure; macro expansion, which generates code at compile time; and proxy-based interception, which wraps existing functions or objects to inject additional behavior without altering their source code [24]. In the context of distributed systems, metaprogramming has proven valuable for introducing cross-cutting concerns—such as logging, monitoring, and fault tolerance—into existing codebases in a modular and transparent manner [25].

2.2. Motivating Example

Modern e-commerce platforms often maintain shopping carts as distributed replicated data structures, synchronizing cart state across geographically distributed replicas to minimize access latency and ensure high availability [5]. In such a system, each replica independently processes local cart updates (e.g., adding or removing items), while the CRDT runtime propagates these changes to all other replicas, eventually converging their states. A Set CRDT is a natural fit for this scenario, as its uniqueness invariant prevents duplicate items from appearing in any cart.

Consider a platform that deploys three replicas across the US, EU, and Asia regions, each maintaining the cart state of its local users. Integrated with the US replica, a recommendation engine puts product suggestions into users’ carts based on their browsing history. As Figure 1 illustrates, a software defect causes the recommendation engine to read from a corrupted user

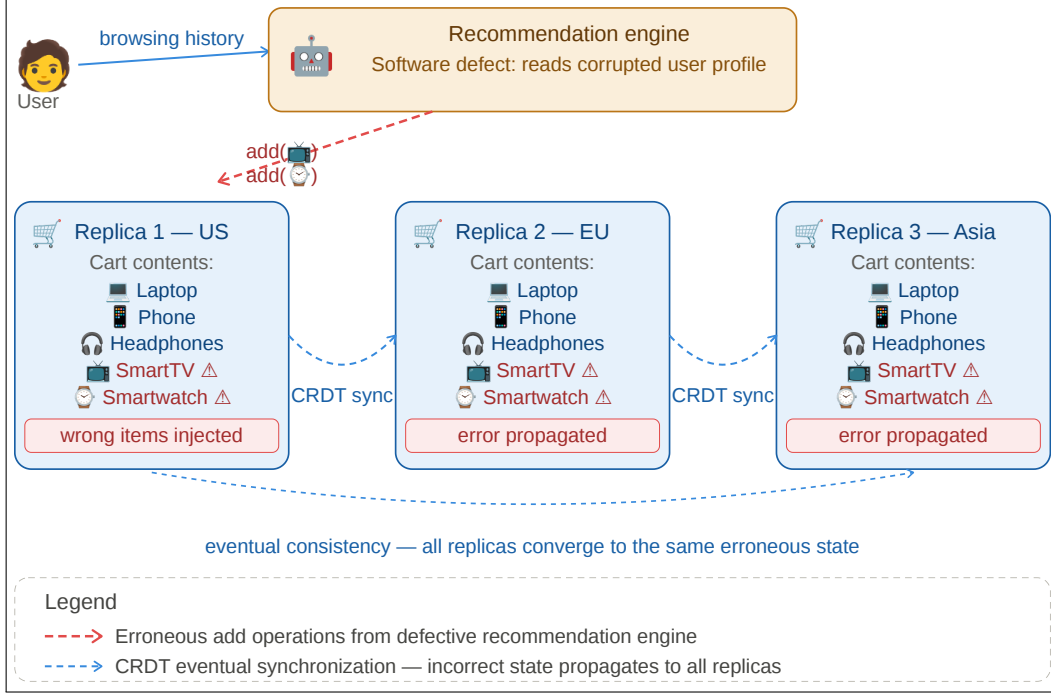


Figure 1: Motivating Scenario: Propagation of Erroneous Updates Across Replicas

profile, injecting wrong items into the cart. In the distributed systems, fault detection is inherently asynchronous: a component may pass local validation checks yet trigger a detectable anomaly only once its downstream effects accumulate across replicas and become observable at the system level [26], for instance, as a statistical deviation in the aggregate CRDT state. This asynchrony can cause erroneous operations to propagate before the fault is detected locally. As a result, each replica converges to a globally consistent but incorrect cart state.

To recover from this situation, the platform must undo the effects of the erroneous update operations. However, the third-party Set CRDT library used by the platform does not provide any built-in undo functionality. A developer tasked with implementing undo might adopt a seemingly straightforward strategy: for each executed update operation, identify its counter-operation (i.e., `remove()` for `add()` and vice versa), record it, and replay the recorded counter-operations in reverse order.

Figure 2 shows why this naive strategy fails. Consider a sequence of six cart operations starting from the initial state $\{Laptop, Phone, Headphones\}$.

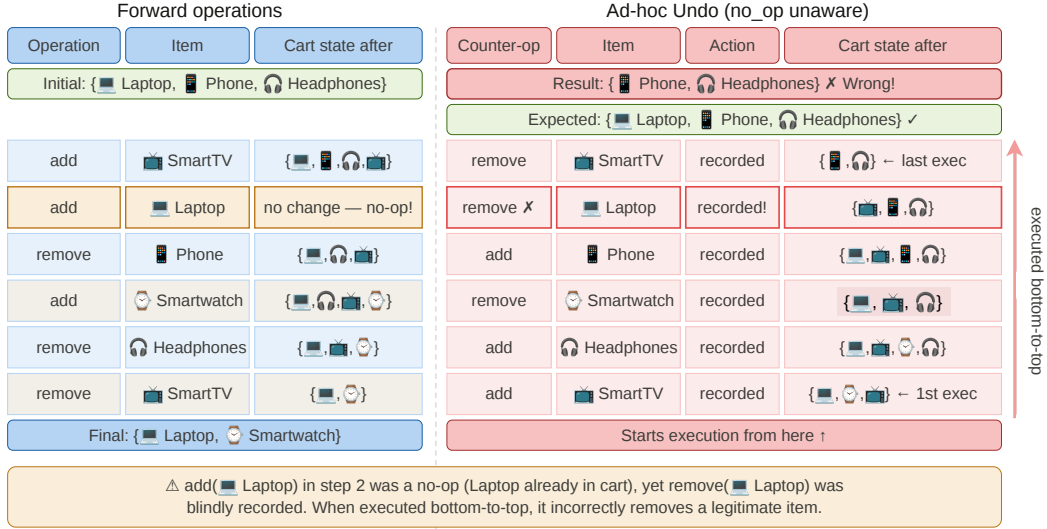


Figure 2: Ad-hoc Undo: Erroneous Cart State Resulting from **no_op** Unawareness

Among these operations, $add(Laptop)$ is a **no_op**: since Laptop is already present in the cart, the Set CRDT’s uniqueness invariant ensures that this operation is discarded, leaving the cart state unchanged. However, if a naive strategy blindly records $remove(Laptop)$ as the counter-operation for this **no_op**, when the undo is executed in reverse order, $remove(Laptop)$ would incorrectly remove a legitimately present item, corrupting the cart state. As a result, such an undo procedure would produce a replicated state of $\{Phone, Headphones\}$, instead of the expected $\{Laptop, Phone, Headphones\}$.

The root cause of this failure is that the naive strategy fails to distinguish between operations that actually modify the CRDT state and those that do not. Only operations that modify the state need to be undone; recording counter-operations for **no_op** (s) inevitably causes incorrect undo behavior. The challenge is to efficiently determine whether a given operation changed the state, without incurring the prohibitive cost of storing and comparing the entire CRDT state before and after each operation.

Beyond correctness, this scenario also highlights the need for undo functionality to be both *automated* and *configurable*. In distributed systems, erroneous updates can propagate across replicas at unpredictable times and in different orders [4]. By the time a user becomes aware of the problem, the corrupted state has already converged system-wide, making timely human-in-the-loop intervention impossible. Moreover, the conditions that necessitate

an undo, such as a statistical anomaly in the replicated state or a threshold violation in sensor readings, are fundamentally data-driven and continuous, thus requiring a monitoring and actuation mechanism that operates at system speed rather than relying on human reaction times [27]. In distributed systems operating at scale, replicas may span geographic regions, faults may be detected asynchronously, and the corrective action’s window may be narrow [28]. The intended model thus enables developers to *pre-configure* when to trigger undo—expressing their expectation declaratively at design time—while the system autonomously detects and actuates undo at runtime. This pre-configured automation model places user agency at configuration time rather than at interaction time, enabling reliable undo at the speed of system operation.

As a solution, the approach presented herein automatically generates the required undo logic for suitable CRDTs and triggers it under configurable conditions, such as a statistical anomaly in the cart contents or an explicit signal from a fault-detection component. In the following section, we detail how CAMEL provides this capability by providing a transparent layer that manages undo logic independently of the underlying CRDT implementation.

3. Solution Approach

To achieve the automation and configurability necessitated by the challenges described in Section 2.2, CAMEL relies on two foundational mechanisms: a lightweight `no_op` detection approach and a declarative metadata configuration model, thus enabling CAMEL to lift existing CRDT libraries with fine-grained, configurable undo functionality, without modifying their source code. In this section, we first present the formal design of these mechanisms and then revisit the motivating example to demonstrate how it addresses the challenges of automated undo.

3.1. *no_op* Indicators

CAMEL’s undo correctness hinges on efficiently determining whether an update operation actually changed the CRDT state. If an operation has no effect (a `no_op`), its counter operation must not be recorded in the Undo Script. Applying it would corrupt the state rather than restore it. Comparing the entire CRDT state before and after every operation would create excessive overhead. State comparison has linear complexity proportional to the number

of elements. Persisting state snapshots for every operation is impractical for large data structures.

CAMEL instead relies on lightweight, data-structure-specific `no_op` indicators, scalar properties that can be evaluated in constant time and that reliably reflect whether the CRDT state has changed. These indicators are specified declaratively in the metadata (as the `no_op` field) and are evaluated at interception time, as described in Section 3.2. We next prove that the chosen indicators are correct for each of the three CRDTs targeted by CAMEL: Counter, Set, and Map.

Definition 3.1 (`no_op` Operation). Let $(S_i, o) \Rightarrow S_j$ denote that operation o applied to a CRDT in state S_i produces state S_j . Operation o is a `no_op` if and only if $S_i = S_j$.

We first introduce formal operation models for each of the three CRDT types, specifying their complete operation sets and the effect of each operation on the CRDT state. These definitions serve as the basis for the exhaustive case analyses at the proof of `no_op` indicators.

Definition 3.2 (Counter CRDT Operation Model). A Counter CRDT maintains a single numeric value $v \in \mathbb{Z}$. Its complete operation set is $\mathcal{O}_C = \{\text{inc}(d), \text{dec}(d)\}$, where $d \in \mathbb{Z}$ is a configurable increment or decrement amount. The effect of each operation on state v is:

- `inc`(d): $v \mapsto v + d$, modifying the state if and only if $d \neq 0$.
- `dec`(d): $v \mapsto v - d$, modifying the state if and only if $d \neq 0$.

Definition 3.3 (Set CRDT Operation Model). A Set CRDT maintains a collection S of unique elements. Its complete operation set is $\mathcal{O}_S = \{\text{add}(e), \text{remove}(e)\}$, where e is an element. The effect of each operation on state S is:

- `add`(e): $S \mapsto S \cup \{e\}$, modifying the state if and only if $e \notin S$; a `no_op` if $e \in S$.
- `remove`(e): $S \mapsto S \setminus \{e\}$, modifying the state if and only if $e \in S$; a `no_op` if $e \notin S$.

Definition 3.4 (Map CRDT Operation Model). A Map CRDT maintains a collection of unique key-value pairs, represented as $M = (K, V)$, where K is

the set of keys and V is the corresponding set of values, with $L = |K|$ denoting the map's size. Its complete operation set is $\mathcal{O}_M = \{\text{put}(k, v), \text{delete}(k)\}$, where k is a key and v is a value. The effect of each operation on state $M = (K, V)$ is:

- **put**(k, v): if $k \notin K$, inserts the pair (k, v) , modifying both K and V (and incrementing L); if $k \in K$ and the existing value $v' \neq v$, updates the mapping to v , modifying V only while K and L remain unchanged; if $k \in K$ and $v' = v$, the state is unchanged (a **no_op**). Note that **put** subsumes both insertion and value modification under a single operation. A key-only mutation—changing a key k to k' while preserving its associated value—is not an atomic operation in this model: it requires two sequential operations, **delete**(k) followed by **put**(k', v), each of which individually modifies both K and L . Consequently, no single operation in $\mathcal{O}_M$ can modify K while leaving both V and L unchanged.
- **delete**(k): if $k \in K$, removes the pair (k, v) , modifying both K and V (and decrementing L); if $k \notin K$, the state is unchanged (a **no_op**).

Several remarks on Definition 3.4 are in order. First, the **put** operation unifies insertion and value modification under a single interface, consistent with standard CRDT practice [5] and the metadata specification presented in Section 3.2. Second, key-only mutations are structurally impossible as single operations in $\mathcal{O}_M$: renaming a key from k to k' while preserving its value v requires **delete**(k) followed by **put**(k', v), meaning any key change necessarily modifies both K and L . This structural property—rather than an imposed restriction—ensures that no single operation in $\mathcal{O}_M$ can leave L and V unchanged while modifying K , which is the key invariant underlying the correctness of the **no_op** indicator in Theorem 3.3.

With these operation models established, we now turn to proving the correctness of the **no_op** indicators for each CRDT type.

Theorem 3.1. *For a Counter CRDT, an update operation $u \in \mathcal{O}_C$ is a **no_op** if and only if the counter's value is unchanged after applying u .*

Proof. Let v_i denote the counter's value before applying u , and v_j the value after; $(v_i, u) \Rightarrow v_j$. By Definition 3.2, the complete operation set $\mathcal{O}_C = \{\text{inc}(d), \text{dec}(d)\}$ with $d \in \mathbb{Z}$ modifies the counter value if and only if $d \neq 0$: **inc**(d) produces $v_j = v_i + d$, and **dec**(d) produces $v_j = v_i - d$, both equal to v_i when $d = 0$. Since the state of a Counter CRDT is fully determined

by its value, $S_i = S_j$ if and only if $v_i = v_j$. By Definition 3.1, u is therefore a **no_op** if and only if its value remains unchanged, which occurs precisely when $d = 0$. $\square$

Theorem 3.2. *For a Set CRDT, an update operation $u \in \mathcal{O}_S$ is a **no_op** if and only if the set's cardinality is unchanged after applying u .*

Proof. Let S and S' denote the set's state before and after applying u , with cardinalities L and L' respectively; $(S, u) \Rightarrow S'$. By Definition 3.3, the complete operation set $\mathcal{O}_S = \{\text{add}(e), \text{remove}(e)\}$ consists of exactly two single-element operations: **add**(e) increases cardinality by one on success ($e \notin S$), and **remove**(e) decreases it by one on success ($e \in S$). If either operation succeeds in modifying the set, $L \neq L'$. Conversely, if $L = L'$ after applying u , the operation did not insert or remove any element—**add**(e) was a **no_op** because $e \in S$, or **remove**(e) was a **no_op** because $e \notin S$ —and therefore $S = S'$. By Definition 3.1, u is a **no_op** if and only if $L = L'$. $\square$

Theorem 3.3. *For a Map CRDT, an update operation $u \in \mathcal{O}_M$ is a **no_op** if and only if both the map's size and its collection of values are unchanged after applying u .*

Proof. Let $M = (K, V, L)$ and $M' = (K', V', L')$ denote the map's state before and after applying u , where K is the key set, V the value collection, and $L = |K|$ the size; $(M, u) \Rightarrow M'$. By Definition 3.4, the complete operation set $\mathcal{O}_M = \{\text{put}(k, v), \text{delete}(k)\}$ covers all permitted map operations. We perform exhaustive case analysis:

- **put**(k, v) with $k \notin K$: modifies both K and V , so $L' = L + 1 \neq L$. Not a **no_op**.
- **put**(k, v) with $k \in K$ and $v' \neq v$: K and L unchanged, but $V' \neq V$. Not a **no_op**.
- **put**(k, v) with $k \in K$ and $v' = v$: K , V , and L all unchanged, so $M = M'$. A **no_op**.
- **delete**(k) with $k \in K$: modifies both K and V , so $L' = L - 1 \neq L$. Not a **no_op**.
- **delete**(k) with $k \notin K$: K , V , and L all unchanged, so $M = M'$. A **no_op**.

By Definition 3.4, a key-only mutation is not an atomic operation in $\mathcal{O}_M$ —any change to K necessarily modifies L as well, since it requires **delete** followed by **put**. This eliminates the possibility that $L = L'$ and $V = V'$ while $K \neq K'$ under any single operation in $\mathcal{O}_M$. Therefore, if $L = L'$ and $V = V'$ after applying u , the exhaustive case analysis above confirms that $M = M'$. By Definition 3.1, u is a **no_op** if and only if $L = L'$ and $V = V'$. $\square$

Theorem 3.4. *If u is a **no_op** and u' is the counter-operation of u , then u' must also be a **no_op** in the post-state; consequently, u' must not be applied to undo u .*

Proof. Let $(S_n, u) \Rightarrow S_{n+1}$ with $S_n = S_{n+1}$, so u is a **no_op** by Definition 3.1. To undo u , the counter-operation u' would need to be applied in state S_{n+1} to recover S_n . Since $S_n = S_{n+1}$, recovering S_n from S_{n+1} via u' requires $(S_{n+1}, u') \Rightarrow S_n = S_{n+1}$, which by Definition 3.1 means u' is itself a **no_op** in S_{n+1} . A **no_op** operation produces no state change, so applying u' neither restores nor corrupts the state. It is therefore unnecessary and must be excluded from the Undo Script to preserve correctness. $\square$

Theorems 3.1–3.4 establish the correctness of the **no_op** indicators used in the metadata specifications that follow. The indicators are data-structure-specific by necessity, as it would be impossible to identify a single property that serves as a universal **no_op** indicator across all CRDT types. However, as the theorems demonstrate, identifying the correct indicator for a given CRDT requires knowledge of its basic state representation and operational semantics, which is a reasonable expectation for developers familiar with the data structure. While the joint size-and-value-collection indicator for Map could be further refined per operation type, for instance, a key-set size check alone suffices to detect a **no_op** for **delete**, the unified indicator is deliberately chosen for its simplicity and uniformity across the metadata specification, avoiding the need for operation-specific branching in the configuration.

3.2. Metadata Specification

A key aspect of CAMEL’s usability is the simplicity of the declarative metadata that developers must provide. The metadata serves as the sole configuration interface between the application developer and CAMEL’s runtime: it specifies the target CRDT operations, their counter-operations, the

`no_op` indicators, and the triggering conditions, without requiring any modification to the underlying CRDT library. We next present the metadata specifications for the three data structures used in our implementation, illustrating how CAMEL’s declarative model accommodates increasingly complex data structures.

Counter. Counter provides a distributed counter with `inc()` and `dec()` operations, each of which increments or decrements the counter’s value by a configurable amount. The `no_op` indicator for a Counter is the `value` field: if the counter’s value is unchanged after an operation, the operation had no effect, and its counter-operation need not be recorded. The following metadata configuration is sufficient to enable CAMEL monitoring Counter CRDT and trigger undo using a deterministic model based on the mean of the tracked values, with a threshold of 3:

```
{"crdt": "Counter",
  "no_op": "value",
  "operations": [
    {"op": "inc", "counter": "dec", "param": "value"},
    {"op": "dec", "counter": "inc", "param": "value"}
  ],
  "trigger": [
    {"model": "det", "procedure": "mean", "threshold": "3"}
  ]}
```

Set. Set maintains a collection of unique elements with `add()` and `remove()` operations. The `no_op` indicator is the `size` field: if the set’s cardinality is unchanged after an operation, the operation was a `no_op`. The metadata below configures CAMEL to use a deterministic trigger based on standard deviation for small sets, switching to a probabilistic trigger via logistic regression once the set size exceeds 10,000 elements:

```
{"crdt": "Set",
  "no_op": "size",
  "operations": [
    {"op": "add", "counter": "remove", "param": "val"},
    {"op": "remove", "counter": "add", "param": "val"}
  ],
  "switch_threshold": "10000",
  "trigger": [
    {"model": "det", "procedure": "std", "threshold": "2"},
    {"model": "prob", "procedure": "lr", "threshold": "0.5"}
  ]}
```

Map. Map maintains a collection of unique $\{\text{key}, \text{value}\} (K \rightarrow V)$ pairs, supporting `put()` and `delete()` operations. The Map configuration is the most involved of the three CRDTs targeted by CAMEL, because the counter-operation for `put(key)` update depends on the runtime state of the data structure: if the key did not exist before the execution of `put(key)` operation, the counter-operation is `delete(key)`; if the key already existed, the counter-operation is `put(key)` with the previous value. CAMEL accommodates this state-dependent branching through the `cond` tag, which specifies a boolean predicate evaluated at interception time. The `no_op` indicator combines two checks—`size` and `get(key)`—to detect both structural and value-level `no_op` (s). The trigger configuration mirrors that of the Set, with standard deviation as the deterministic procedure and logistic regression as the probabilistic fallback, applied once the map exceeds 20,000 entries:

```
{
  "crdt": "Map",
  "no_op": "size & get(key)",
  "cond": "has(key)",
  "operations": [
    {
      "op": "put",
      "counter": "delete",
      "param": "key",
      "cond": "false"
    },
    {
      "op": "put",
      "counter": "put",
      "param1": "key",
      "param2": "prev_value",
      "cond": "true"
    },
    {
      "op": "delete",
      "counter": "put",
      "param1": "key",
      "param2": "value"
    }
  ],
  "switch_threshold": "20000",
  "trigger": [
    {
      "model": "det",
      "procedure": "std",
      "threshold": "2"
    },
    {
      "model": "prob",
      "procedure": "lr",
      "threshold": "0.45"
    }
  ]
}
```

When `cond` evaluates to `false`—meaning the key is being inserted for the first time—the counter-operation is `delete(key)`, requiring only the key as its parameter. When `cond` evaluates to `true`—meaning the `key` already exists—the counter-operation is `put(key, prev_value)`, requiring both the key and the value captured at interception time. The counter-operation for `delete()` is unconditional: `put(key, value)` always restores the deleted entry using the value preserved in the undo stack at the time of deletion.

The three specifications above demonstrate that CAMEL’s declarative model is sufficiently expressive to capture the undo requirements of data structures ranging from a simple scalar counter to a conditional, state-dependent

map. In each case, developers provide only the CRDT-specific knowledge—undo pairs, the `no_op` indicator, and trigger conditions—while CAMEL automatically generates and actuates the required undo functionality. The correctness of CAMEL’s undo behavior depends on the semantic accuracy of the developer-supplied metadata; the implications of misconfigured specifications and the available validation mechanisms are discussed in Section 7.

While CAMEL’s current implementation works for Counter, Set, and Map CRDTs, its declarative model can be adapted to support additional operation-based CRDT types through the same metadata specification approach. Consider, for example, a *List CRDT*, which maintains an ordered sequence of elements with `insert(pos, e)`, `delete(pos)`, and `modify(pos, e)` operations [5]. The `no_op` indicator for a List CRDT would need to combine two checks—`length` and `get(pos)`—analogous to the joint size-and-value indicator used for Map: if both the list’s length and the value at the affected position remain unchanged after an operation, the operation had no effect. This condition accounts for all three List operations: `insert` and `delete` are detected via length change, while `modify(pos, e)`, which leaves length unchanged but updates the element at `pos`, is detected via the value check. The counter-operation pairs follow the same pattern as Map: `insert` is undone by `delete` and vice versa, while `modify(pos, e)` is undone by `modify(pos, eprev)`, where `eprev` is the previous value captured at interception time—analogous to Map’s `put(k, vprev)` counter-operation for value updates. The triggering conditions would likewise be expressed declaratively via the same `trigger` field, requiring no changes to CAMEL’s runtime machinery. The primary challenge in extending CAMEL to List CRDTs—and more broadly to registers, trees, and graphs—lies not in the metadata model, which remains applicable, but in operation-level concerns such as position indexing under concurrent insertions and deletions, which may invalidate recorded positions by the time the Undo Script is actuated. Extending CAMEL to these additional CRDT types and addressing their respective operation-level challenges constitute directions for future work.

3.3. Motivating Example Revisited

Recall the motivating example in Section 2.2, where an ad hoc undo procedure produced an incorrect replicated cart state due to `no_op` unawareness. Applying the Set CRDT metadata specification from Section 3.2—which designates `size` as the `no_op` indicator—CAMEL intercepts each cart operation and evaluates the set’s cardinality before and after execution.

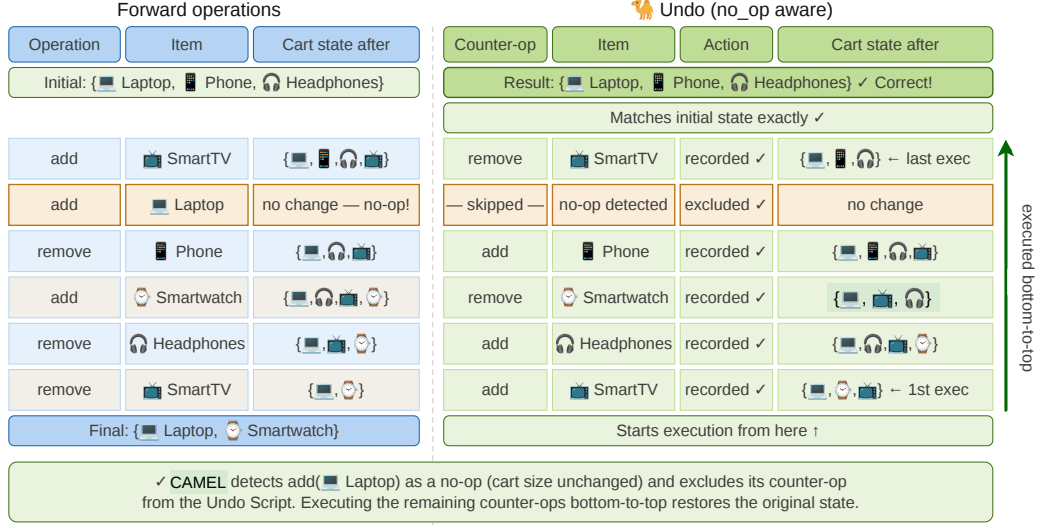


Figure 3: CAMEL’s Undo: Correct Restoration of Initial Cart State via **no_op** Awareness

When `add(Laptop)` leaves the size unchanged, it is classified as a **no_op** by Theorem 3.2 and excluded from the Undo Script. As Figure 3 shows, the resulting **no_op**-aware undo correctly restores the cart to its original state of $\{Laptop, Phone, Headphones\}$, without requiring any modification to the CRDT library source code.

4. CAMEL’s System Design

This section presents the system design of CAMEL, describing its distributed execution model, key components, and their interactions.

4.1. Distributed Execution Model

Because the main contribution of this work is a metaprogramming approach for adding undo functionality to existing CRDT libraries, we briefly outline the distributed execution model that informed our key design decisions. CAMEL is designed to operate with CRDT libraries deployed in a distributed system comprising a collection of uniquely identified replicas that communicate via asynchronous message passing. Replicas may experience crash failures and are assumed to recover by restoring their state from durable storage. At least one replica is assumed to remain operational at all

times to preserve system liveness. The network connecting the replicas may partition temporarily, but is expected to regain connectivity eventually.

These assumptions do not model raw network behavior; rather, they reflect guarantees commonly provided by the transport and middleware layers on which CRDT libraries are typically built [29, 30]. Reliability and deduplication are provided by transport-layer primitives such as TCP [31], and resilient dissemination across partitions is provided by middleware protocols such as gossip [32]. Adopting this abstraction boundary allows us to focus on CAMEL’s architectural contribution rather than on fault-tolerance mechanisms that are orthogonal and well-studied. Relaxing these assumptions would require changes to the underlying communication substrate but would not fundamentally alter CAMEL’s design or its extensibility model. The state of each replica can be modified by its local clients, and these changes are eventually propagated to all other replicas. Upon receiving an update message, a replica performs the necessary actions to synchronize its state with the updates.

4.2. System Components and Their Interactions

Building on the `no_op` detection and metadata configuration mechanisms presented in Section 3, CAMEL implements these concepts via four collaborating components that together provide undo with well-defined, modular component boundaries.

Figure 4 provides an architectural overview of CAMEL. The system accepts two forms of input from the developers: (1) *declarative metadata*, expressed as JSON, which specifies the target CRDT operations, their counter-operations, `no_op` indicators, and triggering conditions; and (2) *annotated application code*, in which developers mark the update operations to be monitored using the higher-order function `undoable([...])`. Based on these inputs, CAMEL’s four components—Update Function Interceptor, Update Manager, Undo Script Generator, and Undo Script Actuator—collaborate to intercept CRDT operations, generate the required Undo Script, and actuate it when required. As CAMEL works with CmRDTs exclusively, the operation-level granularity enables CAMEL to generate fine-grained Undo Script (s) composed of individual counter-operations, rather than reverting the system to a previously checkpointed global state. We next describe the four key components and their interactions.

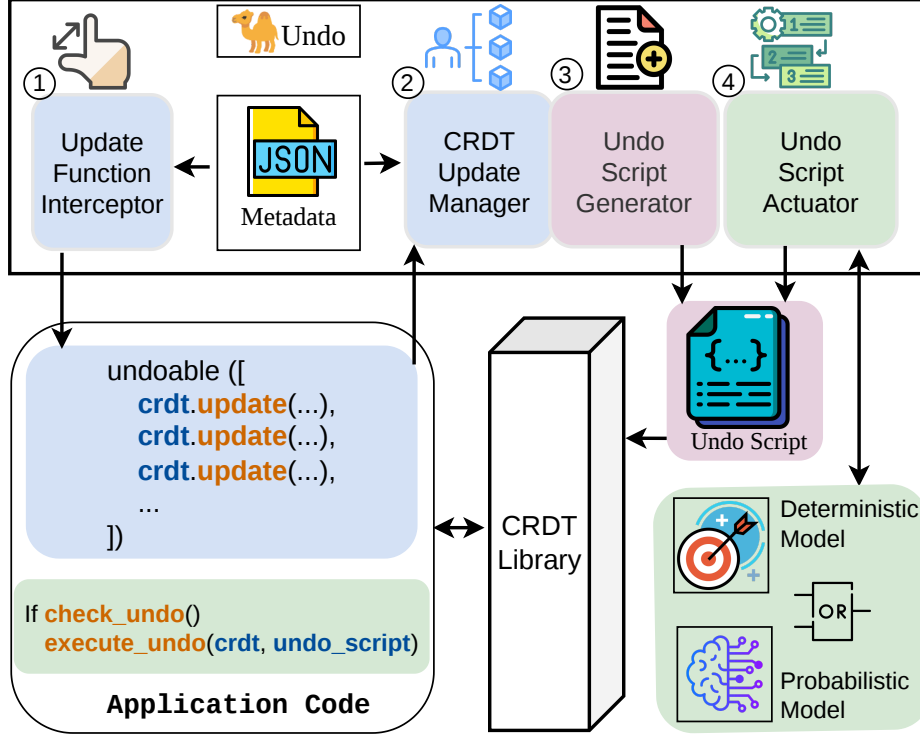


Figure 4: Architectural Overview of CAMEL: Four Components and their Interactions

① Update Function Interceptor

The Update Function Interceptor transparently proxies the target CRDT library’s update operations as they are invoked by the application. Rather than modifying the CRDT library source code, CAMEL generates proxy functions for each target operation specified in the metadata. Each proxy wraps the original function, performing two additional tasks before delegating to the original implementation: (i) it tags the invocation with a Lamport Timestamp (LT) [33] to establish causal ordering among operations, and (ii) it notifies Undo Script Generator to record the corresponding counter-operation, provided the intercepted operation turns out not to be a `no_op`. The use of Lamport Timestamps is a deliberate design choice that underpins CAMEL’s SEC preservation guarantee. By assigning a strictly increasing logical clock value to each intercepted operation, CAMEL establishes a partial causal order over the sequence of update operations within an `undoable` block, which is

embedded in the generated Undo Script and governs how undo operations are synchronized and applied across replicas.

② Update Manager

The Update Manager serves two interdependent roles. First, it *tracks* the update operations invoked within an `undoable(...)` block and coordinates with the Interceptor to monitor which operations have actually changed the CRDT state. Second, it *manages the queuing* of incoming remote synchronization requests while an undoable block is in execution. The queuing mechanism is critical for correctness: CAMEL determines the counter-operation for each intercepted update by inspecting the CRDT state at the moment of execution. If a remote replica’s update were processed concurrently and modified the CRDT state mid-block, the counter-operations already recorded in the Undo Script would have been computed against a stale state, producing an incorrect undo. To prevent this, the Update Manager component holds all incoming remote operations in a *causal queue*, ordered by LT, until the `undoable` block completes. Upon completion—whether or not an undo was actuated—the queued operations are replayed in LT order, ensuring the replica converges to the same state as if no interruption had occurred. Note that concurrent updates that arrive *after* the `undoable` block completes are not subject to staleness concerns: the Undo Script has already been fully constructed against the stable local state captured during block execution, and any subsequent remote updates are applied on top of the post-undo state through the CRDT’s standard synchronization protocol, preserving convergence. This design treats the `undoable` block as a logical transaction: it executes as one indivisible unit from the perspective of external synchronization, while preserving the commutativity and eventual delivery properties required by the underlying CmRDT. Developers mark which CRDT update operations to monitor by wrapping them in the `undoable(...)` construct; while our reference implementation uses a higher-order function—the most idiomatic approach in JavaScript—the same intent could be expressed via annotations or decorators in other languages [34].

③ Undo Script Generator

This component automatically constructs the Undo Script, a causally ordered

sequence of counter-operations that, when executed in reverse, restores the CRDT to its state before the first monitored update operation was applied. The generator’s logic hinges on the `no_op` principle: for each intercepted update operation u , the generator evaluates the `no_op` indicator specified in the metadata to determine whether u actually changed the CRDT state. This `no_op` identification is essential for correctness, as demonstrated in Section 2.2: blindly recording counter-operations for `no_op` operations leads to incorrect undo behavior. Formally, let $\mathcal{U} = \langle u_1, u_2, \dots, u_n \rangle$ denote the sequence of intercepted update operations within an undoable block, each tagged with LT, $LT(u_j)$ at interception time, and let u_j^{-1} denote the counter-operation of u_j as specified in the metadata. The generator produces the Undo Script:

$$\mathcal{U}_S = \langle u_{i_k}^{-1}, \dots, u_{i_2}^{-1}, u_{i_1}^{-1} \rangle,$$

where $\{i_1, i_2, \dots, i_k\} \subseteq \{1, \dots, n\}$ is the subsequence of indices for which u_{i_j} is *not* a `no_op`. Each entry in $\mathcal{U}_S$ is tagged with a fresh LT, $LT(u_{i_j}^{-1}) > LT(u_{i_j})$, ensuring that each undo operation causally succeeds its originating operation on all replicas. The Undo Script is stored on a stack, so actuating it in stack-pop order automatically achieves the required reverse execution. Importantly, because each counter-operation is issued through the standard CRDT update interface and participates in the same LT-based causal order as all other operations, applying CAMEL’s undo procedure does not violate the system’s SEC guarantee, as formally established in Appendix A.

④ Undo Script Actuator

The Undo Script Actuator component has two responsibilities: (i) determine whether the generated Undo Script should be executed, based on the current CRDT state and the configured triggering conditions; and (ii) execute the Undo Script when triggered. To determine whether undo is warranted, CAMEL supports two configurable decision-making models, both of which are specified declaratively via metadata. The *deterministic model* applies a statistical function—such as mean or standard deviation—to the current CRDT data and compares the result against a configured threshold. For example, the metadata directive `{model : det, procedure : std, threshold : 2}` instructs CAMEL to trigger an undo if the standard deviation of a CRDT’s elements exceeds 2. This model produces accurate decisions but can become computationally expensive for large datasets. The *probabilistic model* addresses this

scalability concern by employing an ML classifier trained on CAMEL’s execution history to estimate the probability that undo is needed at a given point. Developers can configure both the probability and size thresholds at which CAMEL switches from the deterministic to the probabilistic model, trading a degree of accuracy for a significant reduction in computational overhead.

CAMEL’s two complementary models are intended to widen its applicability. The deterministic model works well for small-to-medium CRDTs, where its $O(n)$ statistical computation is tractable, enabling precise decisions. However, as the CRDT grows beyond the configured size threshold, the cost of iterating over all elements on every undo check can become prohibitive. So, the probabilistic model bounds this overhead to a fixed cost for classifier inference, regardless of the data structure’s size. This adaptive design is intended to enable CAMEL to maintain both accuracy and scalability across the wide range of CRDT sizes encountered in practice. For scenarios where neither the deterministic nor the probabilistic model is expressive enough, CAMEL provides a custom predicate option: developers can supply an arbitrary boolean predicate function that takes the CRDT instance as input and returns a triggering decision. This option enables encoding application-specific undo conditions that go beyond statistical or ML-based detection. For example, undo may need to be triggered upon violation of a domain-specific invariant, upon a signal from an external fault-detection component, or upon breach of a configurable business rule. Like the deterministic and probabilistic models, the custom predicate is preconfigured at design time and automatically evaluated by CAMEL at runtime, consistent with CAMEL’s preconfigured automation model.

When the actuator determines that undo is required, it executes the Undo Script by popping counter-operations from the stack in reverse order and invoking them against the CRDT. Each counter-operation is issued through the standard CRDT update interface, carrying its associated LT, and is therefore propagated to all other replicas by the CRDT’s own synchronization protocol, ensuring that the undo effect is distributed system-wide rather than confined to the local replica.

We scoped CAMEL’s undo initiation to the replica that executed the original operations within an `undoable` block. Each replica independently maintains its own Undo Script, constructed against its local CRDT state at interception time. This design is deliberate: by scoping undo initiation to the originating replica, CAMEL avoids the coordination overhead and SEC complications that arise when multiple replicas independently attempt to undo

the same operation—a scenario that can lead to conflicting compensating operations and convergence violations. When undo is actuated, its effects propagate system-wide via the CRDT’s standard synchronization protocol, ensuring all replicas converge to the corrected state. Extending CAMEL to support cross-replica undo initiation—where any replica may trigger undo of operations originated elsewhere—introduces non-trivial challenges around concurrent undo coordination and is left as a direction for future work.

5. Implementation

CAMEL is implemented in JavaScript, one of the most widely used languages for developing CRDT libraries [12]. This section describes the engineering realization of CAMEL’s four components and their corresponding algorithms. CAMEL follows a modular layered architecture: it wraps the CRDT library as a black box and provides undo through runtime interception, without coupling to any library-specific internals. Metadata is expressed in JSON, the native configuration format for JavaScript, and CAMEL programmatically invokes Python to execute its statistical and ML routines.

To concretely demonstrate CAMEL’s engineering realization, we applied it to two existing third-party CRDT libraries: `js-delta-crdds` [35] and `crdds` [36]. From `js-delta-crdds`, we configured CAMEL for three data structures, Counter, Set, and Map; from `crdds`, we configured Counter and Set, since `crdds` does not provide a Map implementation. These five configurations serve as the running examples throughout this section and as the subjects of the empirical evaluation in Section 6.

① Update Function Interceptor

The Update Function Interceptor implements the proxy mechanism via *monkey patching* [37]—a JavaScript technique that replaces a function on an existing object at runtime without modifying its source code. For each CRDT update function listed in the metadata, CAMEL replaces the original function reference on the CRDT object with a generated proxy closure that captures the original, records the corresponding undo action, delegates execution, and increments the Lamport clock. Algorithm 1 formalizes this procedure.

`MAKE_PROXY` constructs and returns a new function object—the proxy—whose body captures *origFunc* and *crdt* from the enclosing scope. When the application invokes a monitored CRDT operation, it calls this proxy instead

Algorithm 1 Intercept Update Functions

```
1: procedure INTERCEPT_UPDATE_FUNCTIONS
2:    $crdt\_functions \leftarrow \text{read\_functions}(metaData)$  ▷ Iterate targets in metadata
3:   for all  $func \in crdt\_functions$  do
4:      $params \leftarrow metaData[func.name]$ 
5:      $crdt[func.name] \leftarrow \text{make\_proxy}(crdt, func, params)$  ▷ Apply monkey patching
6:   end for
7: end procedure
8:
9: procedure MAKE_PROXY( $crdt, func, params$ )
10:   $origFunc \leftarrow func$ 
11:   $proxy \leftarrow \lambda(args) :$ 
12:    generate_undo_action( $func.name, args$ )
13:     $origFunc.call(crdt, args)$  ▷ Execute black-box CRDT logic
14:     $LC \leftarrow LC + 1$  ▷ Increment Lamport Clock for causal ordering
15:  return  $proxy$ 
16: end procedure
```

of the original operation. The proxy first notifies the Undo Script Generator via `GENERATE_UNDO_ACTION`, passing $func.name$ as the operation identifier and $args$ as its arguments. It then delegates to the original function via $origFunc.call(crdt, args)$, binding the correct CRDT instance as the execution context. Finally, it increments the Lamport clock LC , tagging the event in the system-wide causal order.

② Update Manager

The Update Manager is realized as a higher-order function `undoable(...)` that accepts an array of CRDT update invocations. Internally, it maintains a *remote operation queue*—a first-in-first-out buffer that holds incoming synchronization messages from other replicas while the `undoable` block is executing. This is implemented by temporarily replacing the CRDT’s message-receive handler with a queuing wrapper; upon block completion, the original handler is restored, and all queued messages are replayed in Lamport Timestamp order. This design isolates the local CRDT state during `undoable` block execution, preventing the stale-state corruption described in Section 4.2.

③ Undo Script Generator

The Undo Script Generator is invoked by the proxy on every intercepted update. It evaluates the `no_op` indicator for the operation and, only if the operation is not a `no_op`, pushes the corresponding counter-operation and its

arguments onto the undo stack. Algorithm 2 describes this procedure.

Algorithm 2 Generate Undo Action

```

1: procedure GENERATE_UNDO_ACTION(funcName, args)
2:   no_op  $\leftarrow$  is_no_op(funcName, args, crdt) ▷ Filter no_op operations
3:   if not no_op then
4:     undoFuncName  $\leftarrow$  metaData[funcName].counter
5:     undo_stack.push({ ▷ Record causal counter-op
6:       name : undoFuncName,
7:       args : args,
8:       lt : LC
9:     })
10:   end if
11: end procedure
12:
13: procedure IS_NO_OP(funcName, args, crdt)
14:   indicator  $\leftarrow$  metaData[funcName].no_op_indicator
15:   before  $\leftarrow$  crdt.evaluate(indicator) ▷ Observe state before & after local execution
16:   crdt[funcName](args)
17:   after  $\leftarrow$  crdt.evaluate(indicator)
18:   return before = after ▷ Equality confirms a no_op
19: end procedure

```

The `IS_NO_OP` procedure reads the `no_op` indicator from the metadata (e.g., `size` for a Set CRDT, `value` for a Counter CRDT), evaluates it before and after executing the function, and returns true if the indicator value is unchanged, thus confirming that the operation did not affect the CRDT state. Each undo stack entry records the name of the counter-operation, the operation arguments, and the current LT at the time of recording. For operations such as Map’s `put(key)`, where the appropriate counter operation depends on the runtime state (i.e., whether the key already existed), the previous value is captured in *args* at recording time and preserved in the stack entry, enabling correct undo without requiring a full state log. The staleness of captured *args* is precluded by the Update Manager’s causal queue mechanism: all incoming remote operations are held in the queue throughout the entire `undoable` block execution, guaranteeing that the local CRDT state remains stable at the moment *args* are captured. Consequently, every counter-operation recorded in the Undo Script is computed against an unmodified local snapshot, and no remote update can invalidate the captured arguments before the block completes.

④ Undo Script Actuator

The Undo Script Actuator implements the decision logic and executes the

Undo Script. Algorithms 3 and 4 formalize its two responsibilities.

Algorithm 3 Check Undo

```

1: procedure UNDO_CHECK(crdt, metaData, customLogic)
2:   if customLogic  $\neq$  null then                                ▷ Prioritize developer-supplied custom predicate
3:     return customLogic(crdt)
4:   end if
5:   params  $\leftarrow$  read_params(crdt, metaData)
6:   model  $\leftarrow$  metaData.trigger.model
7:   if  $|crdt| > metaData.switch\_threshold$  then                    ▷ Scalability: adaptive model switch
8:     model  $\leftarrow$  probabilistic
9:   end if
10:  if model = deterministic then
11:    res  $\leftarrow$  deterministic_model(params, metaData)           ▷ Statistical evaluation
12:    return res > metaData.det_threshold
13:  else
14:    res  $\leftarrow$  probabilistic_model(params, metaData)           ▷ ML-based inference
15:    return res > metaData.prob_threshold
16:  end if
17: end procedure

```

Algorithm 4 Execute Undo Actions

```

1: procedure EXECUTE_UNDO_ACTIONS(crdt, metaData, customLogic)
2:   if not undo_check(crdt, metaData, customLogic) then
3:     undo_stack.clear()                                          ▷ Discard if undo is not warranted
4:     return
5:   end if
6:   while undo_stack  $\neq \emptyset$  do                                ▷ LIFO order for state restoration
7:     action  $\leftarrow$  undo_stack.pop()
8:     undoFunc  $\leftarrow$  crdt[action.name]
9:     undoFunc.call(crdt, action.args)                          ▷ Invoke black-box counter-op
10:    LC  $\leftarrow$   $\max(LC, action.lt) + 1$                             ▷ Preserve SEC via causal update
11:  end while
12:  undo_stack.clear()
13: end procedure

```

UNDO_CHECK first evaluates the developer-supplied `customLogic` predicate, if provided, before consulting the configured statistical or ML-based model. The `customLogic` predicate is an optional boolean function that takes the current CRDT instance as input and returns a triggering decision, allowing developers to encode application-specific undo conditions that cannot be expressed through the built-in models. This predicate governs *when* to trigger undo, and is therefore distinct from the `cond` tag introduced in Section 3.2, which governs *which* counter-operation to record at interception time based on the CRDT’s runtime state. While `cond` supports boolean expressions over built-in CRDT accessor methods, `customLogic` accepts an arbitrary developer-defined function over the full CRDT instance, making it

the appropriate extension point for application-specific triggering logic beyond the built-in models.

When no `customLogic` is provided, `UNDO_CHECK` falls back to the configured model—deterministic or probabilistic—selected based on the current CRDT size. The size-based model switch threshold is evaluated explicitly: if the CRDT’s current size exceeds `metaData.switch_threshold`, the probabilistic model is selected regardless of the default configuration. `EXECUTE_UNDO_ACTIONS` traverses the undo stack via `POP()`, which inherently achieves reverse-order execution. The Lamport clock is updated after each undo operation using the standard max-plus-one rule, ensuring that undo operations participate correctly in the system-wide causal order. The stack is explicitly cleared upon completion.

5.1. Implementation Effort

Table 1 summarizes the implementation effort for CAMEL’s four components and the per-CRDT JSON metadata configurations across the two target libraries.

Table 1: CAMEL’s Implementation Effort.

Component		LOC
Shared infra.	Update Function Interceptor	312
	Update Manager	287
	Undo Script Generator	198
	Undo Script Actuator	243
Subtotal		1,040
Per-CRDT config.	Counter (<code>js-delta-crdts</code>)	28
	Set (<code>js-delta-crdts</code>)	41
	Map (<code>js-delta-crdts</code>)	67
	Counter (<code>crdts</code>)	31
	Set (<code>crdts</code>)	44
Subtotal		211
Total		1,251

The 1,040 LOC comprising CAMEL’s shared infrastructure are written once and reused across all target CRDTs, regardless of the underlying library or data structure. The per CRDT configuration cost of 28–67 LOC reflects the JavaScript glue code and JSON metadata required to specify the undo pairs, `no_op` indicators, and triggering conditions for each data structure. The configuration for Map CRDT is the most involved of the three, as its conditional counter-operation logic must distinguish between adding a new key and modifying an existing one, which requires additional branching in the metadata specification. The total implementation effort of 1,251 LOC demonstrates that CAMEL delivers fully configurable, library-agnostic undo functionality at a non-prohibitive engineering cost.

Note that the difference between the `js-delta-crdts` and `crdts` configurations for the same CRDT type reflects library-specific API differences: `crdts` requires additional boilerplate for CRDT object initialization and function registration that `js-delta-crdts` handles internally, resulting in a small but consistent overhead across all per-CRDT configurations. These differences are incidental to the libraries rather than to CAMEL’s metadata model, which remains identical across both.

6. Evaluation

Our evaluation of CAMEL is driven by the following research questions:

1. **RQ1:** What are the performance characteristics of CAMEL compared to existing undo strategies for CRDT-based distributed systems in terms of latency and memory consumption?
2. **RQ2:** How does CAMEL’s modular layered architecture affect the software quality of undo integration compared to ad hoc and state-of-the-art approaches?
3. **RQ3:** What is the accuracy/performance trade-off between CAMEL’s deterministic and probabilistic decision-making models for undo actuation?

In the following subsections, we first describe our experimental setup. We then present our evaluation results, which answer the aforementioned research questions.

6.1. Experimental Setup

To execute all evaluation scenarios, we established a virtual distributed environment consisting of five replicas. Two replicas operated as isolated processes on a 64-bit Ubuntu 20.04 desktop equipped with 32 GB of RAM and an Intel Core i7 processor. Two additional replicas were hosted under equivalent conditions on a 64-bit Ubuntu 20.04 laptop with 8 GB of RAM and an Intel Core i5 processor. The fifth replica ran on a 32-bit Raspbian 9 Raspberry Pi 3 Model B, equipped with 1 GB of RAM and an ARMv7 Quad Core processor. All five replicas communicated over a high-speed LAN with measured latency below 1 ms. Each evaluation scenario comprised 10K CRDT update operations, with 2K updates issued per replica. Following the update phase, undo operations were initiated, with counts ranging from 100 to 1K and distributed evenly across all five replicas.

To contextualize CAMEL’s performance, we selected two strategies from the literature as comparison points: **GenUndo** [38], a generic undo mechanism for state-based CRDTs based on maintaining a companion undo-state CRDT alongside the original data structure, and **StateRoll** [39], an approach that introduces reversibility into CRDTs through a persistent operation-history log replicated across all nodes. Because neither strategy directly targets JavaScript-based operation-based CRDT libraries, we implemented prototypes of both **GenUndo** and **StateRoll** in JavaScript for our evaluation, following the algorithms and data structures described in their respective publications. These prototypes were then applied to the same workload used to evaluate CAMEL, providing a consistent experimental baseline.

6.2. RQ1: Performance Characteristics

Summary: CAMEL exhibits lower latency and memory consumption than both **GenUndo** and **StateRoll** across all evaluated CRDT data structures and workloads.

To answer RQ1, we measured the average latency and peak memory usage of CAMEL, **GenUndo**, and **StateRoll** across increasing workloads of undo operations. Figures 5 and 6 depict the average latency results for **js-delta-crdts** and **crdts**, respectively. Figures 7 and 8 depict the corresponding peak memory usage.

Across all data structures and both libraries, all three strategies exhibit a consistent trend: as the number of undo operations increases, both average latency and peak memory consumption rise monotonically. This is

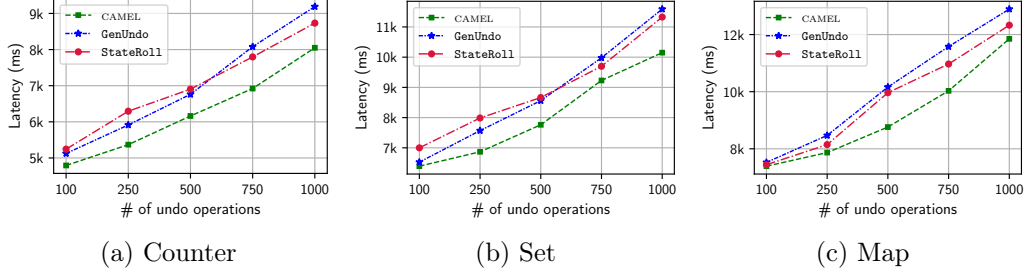


Figure 5: **js-delta-crdts**: Average Latency Across Strategies

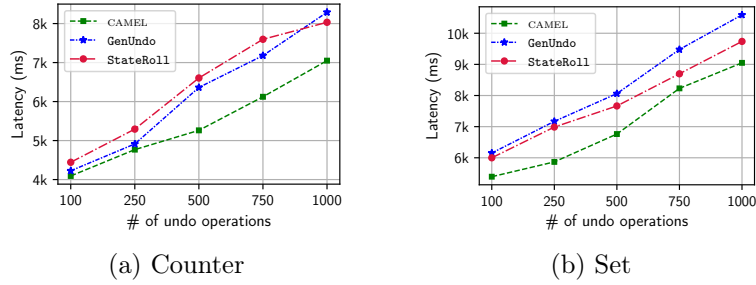


Figure 6: **crdts**: Average Latency Across Strategies

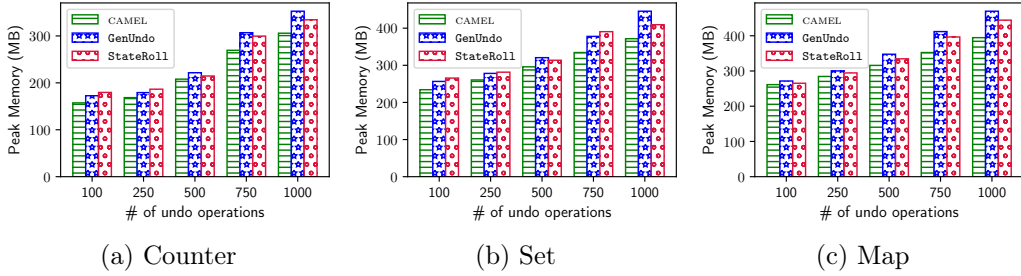


Figure 7: **js-delta-crdts**: Average Peak Memory Usage Across Strategies

expected, as each additional undo operation requires tracking, propagating, and applying compensating operations across replicas.

CAMEL consistently outperforms both **GenUndo** and **StateRoll** in terms of latency and memory consumption across all workload configurations. CAMEL operates directly on operation-based CRDTs, adding only a constant-time `no_op` check and a bounded undo stack per undoable block—both cleared af-

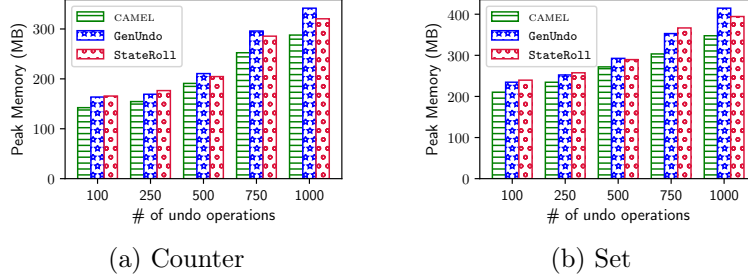


Figure 8: **crdts**: Average Peak Memory Usage Across Strategies

ter execution—and imposes zero overhead on read operations. By contrast, **GenUndo** maintains a monotonically growing companion CRDT whose memory expands with the full operation history and inherits the higher network overhead of state-based synchronization, propagating the entire replica state on every synchronization round rather than the compact encoded operations that **CAMEL** intercepts. **StateRoll**’s operation-history log likewise grows monotonically with every update ever executed, must be replicated to every replica as a companion CRDT, and incurs $O(r)$ overhead on every incoming remote update when checking against the reversed list.

Between the two comparison strategies, **GenUndo** performs better than **StateRoll** at lower undo counts; however, as the number of operations increases, **GenUndo**’s performance degrades more steeply than **StateRoll**’s. This is a direct consequence of **GenUndo**’s state-based synchronization substrate: as more undo operations are issued, the volume of replica state that must be propagated over the network grows proportionally, whereas **StateRoll**’s overhead, though significant, is bounded by its log traversal cost rather than the size of the propagated state.

Quantitatively, across all evaluated configurations, **CAMEL** reduces average latency by approximately 12% compared to **GenUndo** and by approximately 7% compared to **StateRoll**. In terms of peak memory consumption, **CAMEL** achieves approximately 26% and 6% reductions relative to **GenUndo** and **StateRoll**, respectively. The more pronounced memory advantage over **GenUndo** reflects the fundamental cost of state-based synchronization. In contrast, the latency advantage over **StateRoll** grows with operation count due to the compounding overhead of log traversal and replication.

6.3. RQ2: Software Quality Metrics

Summary: CAMEL’s layered, modular architecture demonstrates superior modularity and maintainability, outperforming ad hoc implementations of CAMEL, **GenUndo**, and **StateRoll**.

To answer RQ2, we assessed the software quality of CAMEL’s undo integration relative to three alternatives: a non-service variant of CAMEL, in which undo logic is implemented in an ad-hoc manner (non-service), modifying the CRDT library source code manually (hereafter referred to as $\neg\text{svc.CAMEL}$), and our prototyped implementations of **GenUndo** and **StateRoll**. The goal is to evaluate whether CAMEL’s modular, layered architecture yields measurably better code quality than approaches that either manually embed undo logic or require modifying the underlying CRDT data structure.

We selected four complementary software quality metrics spanning two quality dimensions: modularity and maintainability. For modularity, we collected the Lack of Cohesion in Methods (LCOM) and Coupling Factor (CF) metrics. LCOM quantifies the degree to which the methods of a class or module operate on shared data; lower values indicate that the module’s responsibilities are well-focused and internally coherent, a hallmark of strong separation of concerns [40]. CF measures the proportion of inter-module dependencies relative to the total number of possible dependencies; lower CF indicates fewer cross-module couplings, which in turn improves scalability, testability, and maintainability [41]. For maintainability, we collected Cyclomatic Complexity and Halstead Effort. Cyclomatic Complexity quantifies the number of linearly independent execution paths in the source code by counting decision points such as conditionals and loops [42]; higher values indicate more complex control flow and reduced maintainability. Halstead Effort estimates the cognitive burden of understanding and modifying source code based on the number of distinct operators and operands [43]; lower values correspond to code that is simpler to comprehend and less error-prone to maintain [44].

All metrics were collected exclusively on the undo-related integration code, that is, the code responsible for intercepting CRDT operations, generating the Undo Script, and executing it, rather than on the CRDT library itself. For CAMEL, this corresponds to the proxy, Update Manager, Undo Script Generator, and Undo Script Actuator components. For $\neg\text{svc.CAMEL}$

, this corresponds to the hand-written undo logic embedded directly in the application. For **GenUndo** and **StateRoll**, this corresponds to our prototyped reversal algorithms. Measurements were conducted for each of the five CRDT from the two evaluation subjects: Counter, Set, and Map from **js-delta-crdts**, and Counter and Set from **crdts**. Table 2 reports all four metrics across the four strategies and five data structures.

Table 2: Software Quality Metrics Across Strategies: Lower Scores Indicate Higher Quality

Eval. Subject	CRDT	Strategy	LCOM	CF	Cyclomatic Complexity	Halstead Effort
js-delta-crdts	Counter	CAMEL	8	0.41	11	4821.3
		$\neg$ svc.CAMEL	14	0.67	19	9340.8
		GenUndo	17	0.74	23	14 672.4
		StateRoll	19	0.79	26	18 945.6
	Set	CAMEL	9	0.44	13	5634.7
		$\neg$ svc.CAMEL	16	0.71	21	11 208.3
		GenUndo	19	0.77	25	16 843.9
		StateRoll	22	0.83	29	21 376.5
	Map	CAMEL	11	0.49	16	7102.6
		$\neg$ svc.CAMEL	19	0.76	27	14 891.2
		GenUndo	23	0.84	35	21 534.8
		StateRoll	26	0.88	31	27 649.3
crdts	Counter	CAMEL	8	0.42	12	5013.9
		$\neg$ svc.CAMEL	15	0.68	20	9817.4
		GenUndo	18	0.75	24	15 284.7
		StateRoll	20	0.81	27	19 562.1
	Set	CAMEL	9	0.45	14	5891.4
		$\neg$ svc.CAMEL	17	0.72	22	11 634.8
		GenUndo	20	0.78	26	17 291.3
		StateRoll	23	0.85	30	22 478.6

Across all of these five data structures, CAMEL achieves the best values for every metric. CAMEL’s superior modularity—reflected in consistently lower LCOM and CF values—is a direct consequence of its modular layered architecture. Because CAMEL encapsulates all undo logic within its four well-defined components, each with a concise responsibility, the resulting in-

tegration code exhibits strong internal cohesion and minimal cross-module coupling. The `¬svc.CAMEL` implementation, by contrast, distributes undo logic directly throughout the application code, entangling undo concerns with business logic and resulting in higher LCOM and CF values across all configurations. **GenUndo** and **StateRoll** exhibit the highest LCOM and CF values, as both require maintaining companion data structures—the UState CRDT and the operation-history log, respectively—that are tightly coupled to the internal state representation of the underlying CRDT, introducing structural dependencies that propagate throughout the integration code.

The maintainability advantage of CAMEL is equally reflected in the Cyclomatic Complexity and Halstead Effort measurements. CAMEL intercepts CRDT operations via proxy functions and evaluates a single scalar `no_op` indicator per operation, yielding integration logic with few decision branches and a small vocabulary of operators and operands. The `¬svc.CAMEL` implementation introduces additional branching to manually detect `no_op`, manage the undo stack, and implement triggering conditions inline, thereby increasing control-flow complexity and cognitive burden. **GenUndo**’s integration code carries the highest Cyclomatic Complexity among the four strategies for the Map configuration, reflecting the branching required to traverse join-irreducible states and maintain the UState companion CRDT. **StateRoll** exhibits the highest Halstead Effort values overall, consistent with the breadth of its operation-history log management, DAG traversal logic, and reversed list synchronization, all of which introduce a large vocabulary of operators and operands that compound the cognitive cost of maintaining the integration code. The Map configuration consistently produces the highest metric values across all strategies, reflecting the additional conditional logic required to distinguish between `put(k, v)` applied to a new key, `put(k, v)` applied to an existing key with a different value, and `delete(k)`. CAMEL confines this complexity entirely to its declarative metadata, rather than embedding it in the integration code.

6.4. RQ3: Accuracy/Performance Trade-off of Decision-Making Models

Summary: CAMEL’s deterministic model makes accurate undo decisions at the cost of higher latency on large data structures, whereas the probabilistic model trades some accuracy for lower latency.

To determine whether to actuate an Undo Script, CAMEL supports two

configurable decision-making models: a deterministic model that applies a statistical procedure to the current CRDT state, and a probabilistic model that approximates the same decision using an ML classifier. To our knowledge, we are the first to have experimented with deploying two such complementary models as a configurable triggering mechanism. Hence, we make no broader novelty claims, but rather evaluate the practical trade-off between these two models, enabling developers to decide which choice best fits a given application scenario.

In this context, *accuracy* refers to the agreement between the probabilistic model’s triggering decision and the deterministic model’s ground-truth decision for the same CRDT state. A *correct* decision occurs when the probabilistic model agrees with the deterministic model: either both trigger undo, or neither does. An *incorrect* decision occurs when the two models disagree. This decision is grounded in the observation that the deterministic model, given a precisely configured statistical threshold, always produces the intended triggering outcome; the probabilistic model’s accuracy is therefore measured relative to this baseline.

Experimental methodology. To isolate the performance and accuracy characteristics of the two models independently of network and replication overhead, experiments were conducted on a single replica. More than 1,000 elements were progressively added to the Set and Map CRDT instances. The probabilistic model was trained on 1,000 responses generated by the deterministic model before evaluation; since model training can be performed offline and is a one-time cost, training time is excluded from all latency measurements. The deterministic model was configured to compute the standard deviation of the contained elements against a fixed threshold. These experiments apply only to the Set and Map data structures, as the Counter does not reach the size threshold at which the probabilistic model is warranted under the configured metadata, making the deterministic model the appropriate and sufficient choice for counter-typed CRDTs.

Latency trade-off. Figure 9 compares the average latency of the two models as the number of elements in the CRDT increases.

At smaller data structure sizes, both models exhibit comparable latency. As the number of elements grows, the deterministic model’s latency increases measurably, because computing the standard deviation requires iterating over all contained elements, an $O(n)$ operation whose cost scales linearly with the

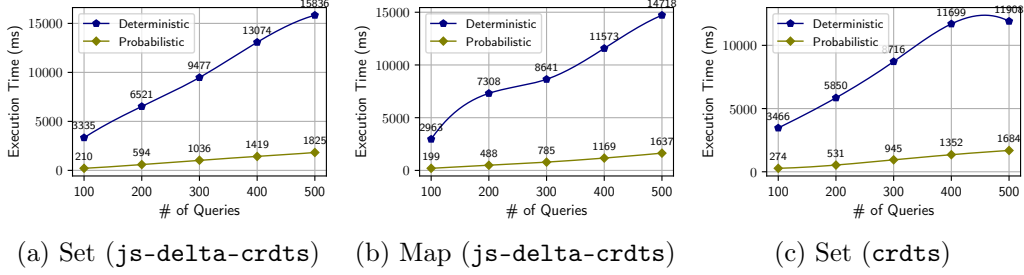


Figure 9: Latency Comparison between Deterministic and Probabilistic Model

data structure size. The probabilistic model, by contrast, reduces the triggering decision to a fixed-cost classifier inference regardless of the number of elements, and its latency advantage over the deterministic model becomes increasingly pronounced as the CRDT grows. This is precisely the scenario for which the probabilistic model is designed: once the data structure exceeds the configured size threshold, CAMEL automatically switches to the probabilistic model to bound the triggering overhead, as specified in the metadata via the `switch_threshold` directive.

Accuracy trade-off. The performance advantage of the probabilistic model comes at the cost of decision accuracy, as shown in Figure 10.

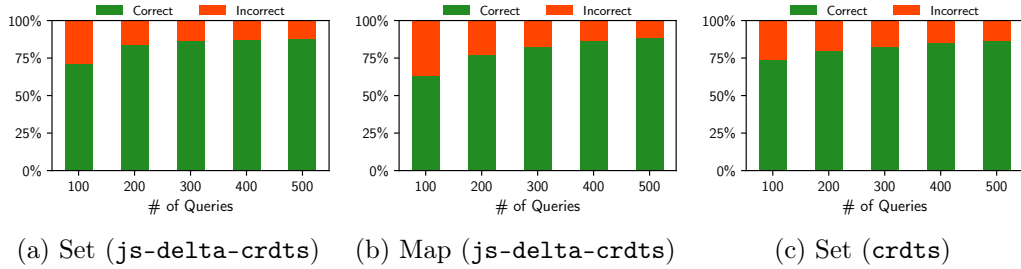


Figure 10: Correct/Incorrect Ratio of the Probabilistic Model

The figure reports the ratio of correct to incorrect decisions produced by the probabilistic model relative to the deterministic model’s ground truth. Initially, when the classifier has accumulated a limited query history, the proportion of incorrect decisions is higher. As the model processes more queries, it continues to refine its decision boundary from accumulated experience, and

the correct-to-incorrect ratio improves consistently across all three evaluated CRDTs. This self-improving behavior is a fundamental property of the logistic regression classifier used in the reference implementation and is consistent with the online learning capabilities of ML models more broadly.

An incorrect decision by the probabilistic model does not compromise the SEC guarantee of the underlying CRDT: it may cause an undo to be triggered when it is not strictly needed, or to be deferred by one query cycle, but the CRDT’s convergence properties are unaffected because counter-operations are issued through the standard CRDT update interface regardless of which model triggers them. The accuracy trade-off is therefore a practical concern about triggering precision rather than a correctness concern about replica convergence.

The choice between the two models is ultimately a developer decision, guided by the size and update frequency of the target CRDT and the application’s tolerance for triggering latency versus triggering precision. CAMEL exposes this choice as a single metadata directive, requiring no changes to application code. A promising direction for future work is the systematic evaluation of alternative classifiers—such as decision trees or neural networks—and adaptive threshold strategies that could further narrow the accuracy gap while preserving the probabilistic model’s latency advantage.

7. Discussion

This section discusses the applicability and limitations of CAMEL, followed by an analysis of the threats to the validity of our evaluation results.

7.1. *Applicability and Limitations*

CAMEL applies to operation-based CRDTs whose update operations come with counter-operations, covering a broad and practically significant class including counters (`inc/dec`), sets (`add/remove`), and maps (`put/delete`). Therefore, CAMEL does not apply to CRDTs that lack counter-operations by design; a prominent example is a grow-only set, whose `add` operation has no inverse. CAMEL employs metaprogramming to intercept CRDT update operations at runtime and generate undo logic. Hence, our approach assumes that the declarative metadata is correct. Indeed, if metadata is configured incorrectly, such as a misconfigured `no_op` indicator or a wrong undo pair, the specified operations will not be undone properly. In the current implementation, CAMEL provides no automated validation of metadata

correctness beyond the basic structural checks of JSON parsing. Hence, diagnosing a misconfigured specification requires manual testing. By applying CAMEL to a controlled CRDT instance with known state transitions, a developer can verify that the `no_op` indicator correctly classifies effective and ineffective operations, and that the recorded counter-operations produce the expected state when executed in reverse. While this testing workflow is straightforward given CAMEL’s declarative metadata model—each field has a well-defined and independently verifiable role—systematically supporting metadata validation, such as automated test generation for `no_op` indicators and undo pair verification, would further reduce the risk of misconfiguration and constitute a direction for future work.

As discussed in Section 3.2, CAMEL’s declarative model is designed to generalize beyond Counter, Set, and Map to additional operation-based CRDT types—such as List, Register, and Tree CRDTs—through the same metadata specification approach, with the primary challenge lying in handling operation-level details specific to each type rather than in CAMEL’s metadata model itself. Extending CAMEL to these additional types constitutes a direction for future work.

Despite these limitations, CAMEL has practical utility because it applies to a reasonable range of deployed CRDTs, treating their libraries as black boxes without modifying their source code. Undo support can be added to a CRDT configuration by providing a straightforward declarative metadata specification, thereby reducing the programming burden on developers. Its configurable triggering layer, which supports deterministic-statistical and probabilistic ML-based models, as well as custom predicates, enables the deployment of undo in settings where human intervention is impractical. Collectively, these properties render CAMEL suitable for adding fine-grained, configurable undo functionality to existing operation-based CRDT libraries.

7.2. Threats to Validity

Our evaluation results are subject to several threats to validity, which we categorize into internal, external, and construct validity threats. Internal validity concerns whether the observed effects can be attributed to the evaluated designs rather than unintended confounding factors, while external validity concerns the generalizability of the findings to other systems and settings [45]. Construct validity concerns whether the selected metrics accurately reflect the properties under study [46].

Internal validity. The primary internal validity threat concerns the prototype implementations of `GenUndo` and `StateRoll`, which serve as comparison points in RQ1 and RQ2. Because neither strategy directly targets JavaScript-based operation-oriented CRDT libraries, we implemented faithful prototypes that follow the algorithms and data structures described in their respective publications [38, 39]. While we exercised care to ensure these prototypes accurately reflect the design intent of each approach, we cannot claim that they represent fully optimized production-quality implementations, and a more expert implementation of either strategy could exhibit different characteristics. We mitigate this threat by grounding our prototypes strictly in the published algorithms, reporting comparison results in relative rather than absolute terms, and providing the architectural reasoning behind the observed differences.

External validity. The evaluation was conducted on five CRDT configurations across two JavaScript CRDT libraries. While these configurations represent widely used CRDT data structures [47], the results may not generalize to all CRDT libraries, programming languages, or application domains without further study. In particular, CAMEL’s performance characteristics may differ in languages with different runtime models or metaprogramming facilities. The distributed environment comprises five heterogeneous replicas communicating over a high-speed LAN, which does not cover wide-area network conditions with higher latency or intermittent connectivity. Under such conditions, the relative performance advantage of CAMEL over `GenUndo`—whose state-based synchronization substrate incurs higher network overhead per synchronization round—would likely be even more pronounced, making our experimental setup a conservative choice that, if anything, understates CAMEL’s advantage in real-world distributed deployments.

Construct validity. The software quality metrics reported in RQ2—LCOM, CF, Cyclomatic Complexity, and Halstead Effort—are well-established measures of modularity and maintainability [40, 41, 42, 43]. However, no single set of metrics captures software quality in its entirety, and other dimensions such as testability and runtime debuggability are not reflected in these measurements. The accuracy metric used in RQ3 is defined as agreement between the probabilistic and deterministic models, which is a reasonable proxy for triggering precision but does not directly measure the business-level impact of missed or spurious undo operations in a deployed system. We acknowledge these as boundaries of the constructs used and note that a more comprehensive quality assessment, including developer studies and long-term

maintainability observations, would further strengthen the evaluation’s empirical foundation.

8. Related Work

Our automated approach to adding undo capability to CRDT libraries draws inspiration from several research areas, including metaprogramming and undo support for CRDT, which we discuss next.

8.1. *Metaprogramming in Distributed Systems*

Metaprogramming—the ability of a program to inspect, generate, or transform code—has been widely applied in distributed systems to introduce cross-cutting concerns with minimal manual intervention. Recent literature underscores the importance of middleware transparency, achieved through metaprogramming techniques that decouple core application logic from systemic requirements such as synchronization and state management [48]. In dynamic execution environments, particularly within the JavaScript ecosystem, the standardization of runtime reflection primitives has enabled the sophisticated interception of object behavior without altering underlying class definitions [22]. This shift is further supported by a growing emphasis on declarative configuration management, in which metadata-driven abstractions enable distributed systems to adapt their state-transition logic dynamically without hard-coded recovery paths [49]. Moreover, researchers have successfully applied runtime instrumentation as a non-invasive means of enhancing reliability in distributed microservices, demonstrating that complex reliability features can be introduced without modifying the original codebase [50]. Within the domain of self-adaptive systems, reflection-based adaptation has been identified as a viable approach to managing consistency protocols and tailoring behavior to changing network conditions [51]. Metaprogrammable frameworks have demonstrated significant potential in automating fault recovery for replicated state machines, treating recovery and roll-back operations as first-class, programmable entities [52]. While CAMEL focuses on dynamic runtime adaptation, it is rooted in the concepts of classical distributed systems where Aspect-Oriented Programming (AOP) was widely applied as a particularly influential metaprogramming paradigm, offering convenient abstractions for weaving additional behavior into existing codebases without modifying their source code [53].

CAMEL draws on this tradition by using proxy-based interception to augment existing CRDT libraries with runtime undo functionality, without modifying their source code. Rather than relying on AOP facilities, CAMEL introduces its own declarative metadata as the configuration interface and leverages language-provided runtime adaptation facilities, e.g., monkey patching in JavaScript, to automatically generate and actuate undo logic.

8.2. *Undo for CRDT-Based Systems*

Enhancing CRDTs with undo capability has become a prominent research topic, driven by the practical need to recover from erroneous updates in eventually consistent distributed systems. The existing literature spans a spectrum from highly customized CRDT designs with built-in undo, to generic approaches that augment existing CRDTs, to library-independent metaprogramming approaches such as CAMEL.

Undo in collaborative editing has been studied extensively. Representative contributions address selective undo of string-wise operations [54], undo applicable to any number of operations rather than only the most recent ones [55], formally proved “undo anywhere, anytime” functionality [56], algorithms satisfying neutrality and forward transposition conditions [57], selective undo with operational transformation consistency control [58], generic algorithms for valid undo operation generation [59], and the Yjs JavaScript framework for collaborative editing with a domain-specific undo/redo manager [60]. While these contributions are significant, they are largely internally-coupled, requiring the undo logic to be deeply integrated into the CRDT’s specific state-transition and convergence properties. Consequently, they lack a library-agnostic framework to augment existing third-party CRDT implementations without invasive modifications to their underlying source code.

Two works served as the primary comparison points in our evaluation. **GenUndo** [38] presents a generic undo mechanism for state-based CRDTs that maintains a companion UState CRDT alongside the original data structure, using undo lengths to capture the semantics of concurrent operations. **StateRoll** [39] introduces reversible CRDTs (rCRDTs) as extensions of existing CRDTs, with an additional reverse operation in their interfaces, supported by a persistent operation-history log. Both works make important contributions but differ from CAMEL in fundamental ways, as summarized in Table 3.

CAMEL is the only approach that requires neither modification of the CRDT library source code nor the design of a new augmented CRDT type.

Table 3: Comparative Analysis of CAMEL Against State-of-the-art Undo Approaches for CRDT-based Systems

Dimension	CAMEL	GenUndo	StateRoll
CRDT Family	Operation-based (CmRDTs) exclusively	State-based (CvRDTs) delta-state focus	Both state-based and operation-based
Undo Mechanism	Explicit Undo Script: ordered counter-operations	State transformation via filtering functions	Compensation functions based on value differences
no_op Handling	Explicit formal detection via scalar indicators	Implicit: handled through empty state deltas	Implicit: based on value difference ($t_k = 0$)
Library Modification	None: transparent interception via meta-programming	High: requires new augmented CRDT types	High: requires new rCRDT types and interface extensions
Triggering	Automated: deterministic and probabilistic models	Manual: left to application logic	Manual: left to application logic
Memory Footprint	Bounded: stack is cleared after block execution	Moderate: buffered state transformations	High: persistent, ever-growing op-history logs
Software Quality	High: Modular architecture; high cohesion, low coupling	Reduced: High coupling via state-delta mapping	Low: High complexity; interface bloat

By treating existing CRDT libraries as unmodified black boxes, CAMEL addresses the significant implementation burden imposed by **GenUndo** and **StateRoll**, both of which require developers to design new augmented data types or manually extend their internal interfaces with compensation logic. While **GenUndo** focuses on the formal semantics of concurrent state-based undo and **StateRoll** introduces a generalized log-based reversal framework, CAMEL shifts the operational complexity to a declarative meta-programming layer. This shift enables automated, model-driven triggering, a capability that could be beneficial to large-scale autonomous systems where human-initiated undo is impractical.

Architecturally, CAMEL optimizes for resource-constrained environments by replacing the high memory overhead of **StateRoll**’s persistent history logs with a bounded undo stack. Furthermore, it provides the explicit **no_op** detection required for operation-based systems, a requirement that **GenUndo** addresses only implicitly through state deltas. Unlike the invasive modifications required by state-of-the-art approaches—which lead to high coupling and interface bloat—CAMEL maintains a modular, layered separation of concerns, significantly improving software quality metrics such as modularity and cohesion.

9. Conclusion

We have presented CAMEL, a metaprogramming-based approach that augments existing operation-based CRDT libraries with automated, configurable undo functionality without modifying their source code, and have demonstrated it on Counter, Set, and Map CRDTs spanning scalar, collection, and associative data structures. CAMEL intercepts CRDT update operations at runtime via proxy-based instrumentation, generates a causally ordered Undo Script of counter-operations guided by a **no_op** detection mechanism, and actuates it based on configurable decision-making models, either deterministic, driven by statistical functions, or probabilistic, driven by ML. All undo logic is derived from compact, declarative metadata specifications, leaving the underlying CRDT library unchanged. Our evaluation demonstrates CAMEL’s superior performance characteristics compared to two state-of-the-art approaches. Furthermore, CAMEL’s modular layered architecture exhibits superior software quality metrics.

Our future work plans include extending CAMEL to additional CRDT data structures—such as List, Register, and Tree CRDTs—and libraries, developing systematic tooling support for metadata validation and debugging, investigating the expressive power of declarative metadata to add advanced functionality to CRDTs, exploring richer decision-making models for undo actuation, and examining whether CAMEL’s metaprogramming-based design generalizes to other cross-cutting concerns in distributed replicated systems.

Data Availability

We make CAMEL’s source code and its evaluation artifacts available at <https://doi.org/10.5281/zenodo.19228545>.

Appendix A. Strong Eventual Consistency (SEC) Preservation Under CAMEL’s Undo Operations

Operation-based CRDTs (CmRDTs) guarantee SEC for their native update operations [5]. CAMEL introduces additional operations into the system—specifically, the counter-operations constituting the Undo Script—that are not part of the CRDT’s original interface. The question we answer here is therefore not whether the CRDT itself satisfies SEC, which is already established by its design, but whether CAMEL’s undo operations, injected into an already SEC-compliant system, preserve this guarantee. In particular, we must show that replicas that receive and apply undo operations still converge to the same state and that eventual delivery is not compromised.

Eventual delivery. Let $\mathcal{R} = \{R_1, \dots, R_m\}$ be a set of replicas executing a CmRDT that satisfies SEC. Each replica maintains a Lamport clock LC_i , updated on local events and on message receipt as:

$$LC_i \leftarrow \max(LC_i, LC_{\text{sender}}) + 1$$

following the standard rule of Lamport [33]. This induces a partial causal order $\rightarrow$ over all events in the system: $e \rightarrow f$ iff e causally precedes f along some chain of local executions and message receipts. Two events are *concurrent*, written $e \parallel f$, iff neither $e \rightarrow f$ nor $f \rightarrow e$.

Let $B = \langle u_1, u_2, \dots, u_n \rangle$ be the sequence of CRDT update operations executed within an `undoable` block at replica R_i , each tagged by the Interceptor with timestamp $LT(u_j)$ at invocation time. For each non-`no_op` operation u_{i_j} in B , let $u_{i_j}^{-1}$ denote its counter-operation as specified in the metadata. The Undo Script generated by CAMEL is:

$$\mathcal{S}(B) = \langle u_{i_k}^{-1}, \dots, u_{i_1}^{-1} \rangle, \quad \text{where } LT(u_{i_j}^{-1}) > LT(u_{i_j}) \text{ for all } j.$$

The strict inequality on timestamps is enforced by the Interceptor at generation time, ensuring that every counter-operation causally succeeds its originating operation in the system-wide partial order. Let Q_i denote the set of remote operations received by R_i during the execution of B and held in the Update Manager’s causal queue, ordered by Lamport Timestamp. These operations are not applied to the local CRDT state until B completes; upon completion, Q_i is replayed in LT order.

Theorem Appendix A.1. *Let $\mathcal{C}$ be a CmRDT satisfying SEC. The introduction of CAMEL’s Undo Script operations into $\mathcal{C}$ preserves SEC.*

Proof. We show that both SEC conditions—eventual delivery and convergence—continue to hold after CAMEL introduces undo operations.

Eventual delivery. CAMEL’s Interceptor proxies each CRDT update operation and delegates to the original CRDT function. Since $\mathcal{C}$ satisfies eventual delivery for all operations submitted through its interface, the intercepted operations $u_j \in B$ are delivered to all replicas by $\mathcal{C}$ ’s own propagation mechanism. The counter-operations in $\mathcal{S}(B)$ are likewise issued through the same standard CRDT interface—they are, from the library’s perspective, ordinary client updates. Therefore, $\mathcal{C}$ ’s propagation mechanism delivers them to all replicas as well. The causal ordering constraint $LT(u_{i_j}^{-1}) > LT(u_{i_j})$ guarantees that no replica applies $u_{i_j}^{-1}$ before u_{i_j} , preserving causal consistency across the delivery order. Hence, every operation introduced by CAMEL is eventually delivered to all replicas, and this holds for eventual delivery.

Convergence. Let $\mathcal{O}$ be the complete set of operations across the system after B completes: $\mathcal{O}$ includes the operations in B , the Undo Script $\mathcal{S}(B)$, and all operations in Q_i together with any operations issued by other replicas concurrently or subsequently. We claim that any two replicas that deliver the same set $\mathcal{O}$ converge to the same state.

The Update Manager ensures that no operation in Q_i is applied to the local CRDT state while B is in execution. This means R_i processes B against a stable, unmodified snapshot of the CRDT state, yielding a deterministic Undo Script $\mathcal{S}(B)$ whose counter-operations are computed against that snapshot. Upon completion of B and any undo actuation, R_i replays Q_i in LT order. Since the operations in Q_i are concurrent with B —by construction, as they arrived during B ’s execution—the commutativity of $\mathcal{C}$ guarantees that replaying Q_i after B at R_i yields the same final state as any replica R_j that delivers the same operations in a different causally consistent order. The counter-operations in $\mathcal{S}(B)$ are propagated to all replicas by $\mathcal{C}$ ’s protocol and applied there in causal order, consistent with their Lamport Timestamps. Because $\mathcal{C}$ guarantees convergence for any causally consistent delivery ordering of its operations, and because every operation in $\mathcal{O}$ —original update, counter-operation, or queued remote operation—participates in the same causal order, any two replicas delivering $\mathcal{O}$ converge to the same state. Convergence, therefore, holds. Since both eventual delivery and convergence hold, CAMEL’s undo operations preserve the SEC of $\mathcal{C}$. $\square$

References

- [1] P. Mondal, E. Tilevich, Undoing CRDT Operations Automatically, in: 2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), IEEE, 2023, pp. 246–251.
- [2] B. Krishnamachari, S. Kapadia, Data replication and scheduling for content availability in vehicular networks, University of Southern California, 2007.
- [3] D. Rotem, S. Seshadri, L. M. Bernardo, Data replication and delay balancing in heterogeneous disk systems, in: WDAS, 1998.
- [4] V. B. Gomes, M. Kleppmann, D. P. Mulligan, A. R. Beresford, Verifying strong eventual consistency in distributed systems, Proceedings of the ACM on Programming Languages 1 (2017) 1–28.
- [5] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski, Conflict-free replicated data types, in: Stabilization, Safety, and Security of Distributed Systems: 13th International Symposium, SSS 2011, Grenoble, France, October 10-12, 2011. Proceedings 13, Springer, 2011, pp. 386–400.
- [6] C. Baquero, P. S. Almeida, A. Shoker, Pure operation-based replicated data types, arXiv preprint arXiv:1710.04469 (2017).
- [7] M. Kleppmann, A. Bieniusa, M. Shapiro, General-purpose crdt libraries, 2020. <https://crdt.tech/implementations>.
- [8] P. Bailis, A. Ghodsi, Eventual consistency today: Limitations, extensions, and beyond: How can applications be built on eventually consistent infrastructure given no guarantee of safety?, Queue 11 (2013) 20–32.
- [9] M. Capra, R. Peloso, G. Maserà, M. Ruo Roch, M. Martina, Edge computing: A survey on the hardware requirements in the internet of things world, Future Internet 11 (2019) 100.
- [10] Y. Zhang, M. Weidner, H. Miller, Programmer Experience When Using CRDTs to Build Collaborative Webapps: Initial Insights, in: 13th annual workshop on the intersection of HCI and PL (PLATEAU), 2023. doi:<https://doi.org/10.1184/R1/22277341>.

- [11] J. Rubin, Cloned product variants: From ad-hoc to well-managed software reuse, University of Toronto (Canada), 2014.
- [12] J. Ivarsson, Real-time collaborative editing using CRDTs, 2019. <https://www.diva-portal.org/smash/get/diva2:1304659/FULLTEXT01.pdf>.
- [13] D. Rambabu, A. Govardhan, Survey on data replication in cloud systems., *Web Intell.* 22 (2024) 83–109.
- [14] P. Mondal, E. Tilevich, Cross-platform integration and extension of replicated data libraries in distributed systems, *Journal of Parallel and Distributed Computing* (2026) 105263. doi:<https://doi.org/10.1016/j.jpdc.2026.105263>.
- [15] M. Kleppmann, A. R. Beresford, A conflict-free replicated JSON datatype, *IEEE Transactions on Parallel and Distributed Systems* 28 (2017) 2733–2746.
- [16] C. Baquero, P. S. Almeida, A. Cunha, C. Ferreira, et al., Composition in state-based replicated data types, *Bulletin of EATCS* 3 (2017).
- [17] M. Shapiro, B. Kemme, Eventual consistency, in: *Encyclopedia of Database Systems*, Springer, 2017, pp. 1–2.
- [18] S. Burckhardt, A. Gotsman, H. Yang, Understanding eventual consistency, Microsoft Research, Redmond, WA, USA, Tech. Rep. MSR-TR-2013–39 (2013) 12.
- [19] S. Burckhardt, et al., Principles of eventual consistency, *Foundations and Trends® in Programming Languages* 1 (2014) 1–150.
- [20] A. Bouajjani, C. Enea, J. Hamza, Verifying eventual consistency of optimistic replication systems, in: *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2014, pp. 285–296.
- [21] W. Tansey, E. Tilevich, Efficient automated marshaling of C++ data structures for MPI applications, in: *2008 IEEE International Symposium on Parallel and Distributed Processing*, 2008, pp. 1–12. doi:[10.1109/IPDPS.2008.4536307](https://doi.org/10.1109/IPDPS.2008.4536307).

- [22] Y. Lilis, A. Savidis, A survey of metaprogramming languages, *ACM Computing Surveys (CSUR)* 52 (2019) 1–39.
- [23] N. Stucki, A. Biboudis, M. Odersky, A practical unification of multi-stage programming and macros, in: *Proceedings of the 17th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, 2018, pp. 14–27.
- [24] The Apache Software Foundation, The Apache Groovy programming language – runtime and compile-time metaprogramming, 2023. URL: <https://groovy-lang.org/metaprogramming.html>.
- [25] B. Donkervoet, G. Agha, Reflecting on aspect-oriented programming, metaprogramming, and adaptive distributed monitoring, in: *FMCO*, Springer, 2006, pp. 246–265.
- [26] Y. Yang, Y. Wu, K. Pattabiraman, L. Wang, Y. Li, How far have we come in detecting anomalies in distributed systems? an empirical study with a statement-level fault injection method, in: *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, IEEE, 2020, pp. 59–69.
- [27] C. Zhao, J. Chen, H. Jing, Condition-driven data analytics and monitoring for wide-range nonstationary and transient continuous processes, *IEEE Transactions on Automation Science and Engineering* 18 (2020) 1563–1574.
- [28] F. Boem, R. Carli, M. Farina, G. Ferrari-Trecate, T. Parisini, Distributed fault detection for interconnected large-scale systems: A scalable plug & play approach, *IEEE Transactions on Control of Network Systems* 6 (2018) 800–811.
- [29] M. G. Merideth, A. Iyengar, T. Mikalsen, S. Tai, I. Rouvellou, P. Narasimhan, Thema: Byzantine-fault-tolerant middleware for web-service applications, in: *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*, IEEE, 2005, pp. 131–140.
- [30] K. Nagar, S. Jagannathan, Automated parameterized verification of CRDTs, in: *Computer Aided Verification: 31st International Conference, CAV 2019, New York City, NY, USA, July 15–18, 2019, Proceedings, Part II* 31, Springer, 2019, pp. 459–477.

- [31] J. Postel, Transmission control protocol, Technical Report, 1981.
- [32] Ö. Babaoğlu, A. Schiper, On group communication in large-scale distributed systems, *ACM SIGOPS Operating Systems Review* 29 (1995) 62–67.
- [33] L. Lamport, Time, clocks, and the ordering of events in a distributed system, *Commun. ACM* 21 (1978) 558–565. URL: <https://doi.org/10.1145/359545.359563>. doi:10.1145/359545.359563.
- [34] A. Osmani, Learning JavaScript design patterns, " O'Reilly Media, Inc.", 2023.
- [35] P. Teixeira, Delta state-based CRDTs in JavaScript, 2018. <https://github.com/peer-base/js-delta-crdts>.
- [36] R. Littauer, A library of conflict-free replicated data types for javascript, 2016. <https://github.com/orbitdb/crdts>.
- [37] Y. Wang, K. S. Cheng, M. Song, E. Tilevich, A declarative enhancement of javascript programs by leveraging the java metadata infrastructure, *Science of Computer Programming* 181 (2019) 27–46.
- [38] W. Yu, V. Elvinger, C.-L. Ignat, A generic undo support for state-based CRDTs, in: *OPODIS 2019-Proceedings of 23rd International Conference on Principles of Distributed Systems*, 2019.
- [39] Y. Mao, Z. Liu, H.-A. Jacobsen, Reversible conflict-free replicated data types, in: *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, 2022, pp. 295–307.
- [40] M. Hitz, B. Montazeri, Measuring coupling and cohesion in object-oriented systems, na, 1995.
- [41] F. B. Abreu, R. Esteves, M. Goulão, The design of Eiffel Programs: Quantitative evaluation using the MOOD metrics, in: *Proceedings of TOOLS'96*, Citeseer, 1996.
- [42] G. K. Gill, C. F. Kemerer, Cyclomatic complexity density and software maintenance productivity, *IEEE transactions on software engineering* 17 (1991) 1284–1288.

- [43] T. Hariprasad, G. Vidhyagaran, K. Seenu, C. Thirumalai, Software complexity analysis using Halstead metrics, in: 2017 International Conference on Trends in Electronics and Informatics (ICEI), IEEE, 2017, pp. 1109–1113.
- [44] M. Alfadel, A. Kobilica, J. Hassine, Evaluation of Halstead and Cyclomatic complexity metrics in measuring defect density, in: 2017 9th IEEE-GCC Conference and Exhibition (GCCCE), IEEE, 2017, pp. 1–9.
- [45] J. Siegmund, N. Siegmund, S. Apel, Views on internal and external validity in empirical software engineering, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, volume 1, IEEE, 2015, pp. 9–19.
- [46] P. Ralph, E. Tempero, Construct validity in software engineering research and software metrics, in: Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018, 2018, pp. 13–23.
- [47] N. Saquib, C. Krintz, R. Wolski, Log-based crdt for edge applications, in: 2022 IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2022, pp. 126–137.
- [48] J. K. Cross, D. C. Schmidt, Meta-programming techniques for distributed real-time and embedded systems, in: Proceedings of the Seventh IEEE International Workshop on Object-Oriented Real-Time Dependable Systems.(WORDS 2002), IEEE, 2002, pp. 3–10.
- [49] I. Lagartos, J. M. Redondo, F. Ortin, Efficient runtime metaprogramming services for java, *Journal of Systems and Software* 153 (2019) 220–237.
- [50] Z. Wu, J. Wang, Q. Qi, M.-G. Shu, R. Chu, J.-B. Li, J. Jin, D. Chen, Flowrca: enhancing microservice reliability with non-invasive root cause analysis, in: 2024 IEEE International Conference on Web Services (ICWS), IEEE, 2024, pp. 1251–1258.
- [51] R. El Ballouli, Modeling self-configuration in Architecture-based self-adaptive systems, Ph.D. thesis, Université Grenoble Alpes, 2019.

- [52] P. B. Ta, V. Mafeni, Y. Kim, A design of workflow-based automated failure recovery framework in iot edge environment, *IEEE Access* (2025).
- [53] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, J. Irwin, Aspect-oriented programming, in: *ECOOP'97—Object-Oriented Programming: 11th European Conference Jyväskylä, Finland, June 9–13, 1997 Proceedings* 11, Springer, 1997, pp. 220–242.
- [54] W. Yu, L. André, C.-L. Ignat, A crdt supporting selective undo for collaborative text editing, in: *IFIP International Conference on Distributed Applications and Interoperable Systems*, Springer, 2015, pp. 193–206.
- [55] S. Martin, P. Urso, S. Weiss, Scalable xml collaborative editing with undo: (short paper), in: *OTM Confederated International Conferences" On the Move to Meaningful Internet Systems"*, Springer, 2010, pp. 507–514.
- [56] S. Weiss, P. Urso, P. Molli, Logoot-undo: Distributed collaborative editing system on p2p networks, *IEEE transactions on parallel and distributed systems* 21 (2010) 1162–1174.
- [57] J. Ferrié, N. Vidot, M. Cart, Concurrent undo operations in collaborative environments using operational transformation, in: *OTM Confederated International Conferences" On the Move to Meaningful Internet Systems"*, Springer, 2004, pp. 155–173.
- [58] B. Shao, D. Li, N. Gu, An algorithm for selective undo of any operation in collaborative applications, in: *Proceedings of the 2010 ACM International Conference on Supporting Group Work*, 2010, pp. 131–140.
- [59] C. Sun, Undo as concurrent inverse in group editors, *ACM Transactions on Computer-Human Interaction (TOCHI)* 9 (2002) 309–361.
- [60] K. Jahns, Yjs: A crdt framework with a powerful abstraction of shared data, 2014. <https://github.com/yjs/yjs>.